



PUC Minas

SEGURANÇA NO SDLC

Edgar Paes



PUC Minas

SECURITY SDLC: PLAN

OBJETIVO

Compreender os principais conceitos primários e fundamentais que envolvem a fase de Planejamento do SDLC (Software Development Life Cycle)

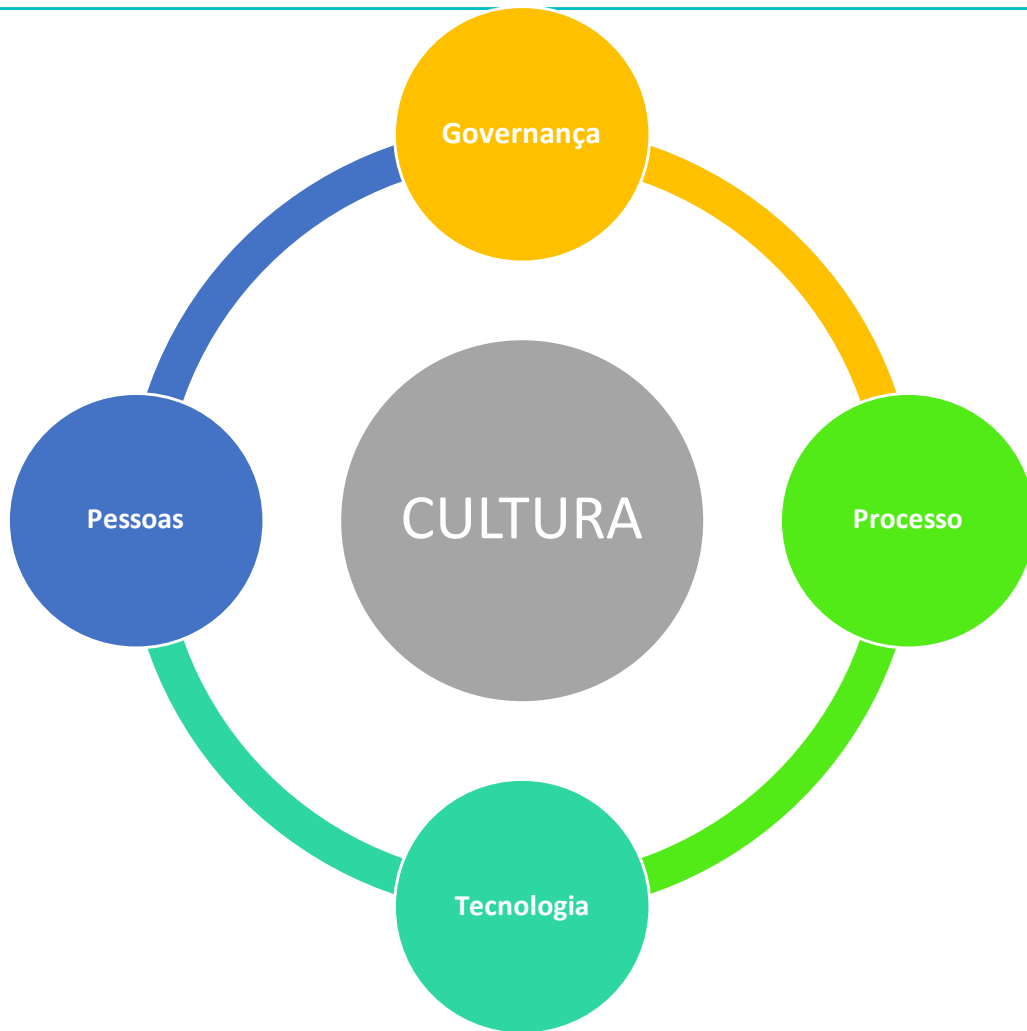
FASE DE PLAN

- Esta fase caracteriza-se pelas definições de alto nível de como o produto será implantado na organização e devido planejamento
- Define os requisitos de segurança de forma abrangente, como a segurança é incorporada ao SDLC, aspectos de cultura no contexto de segurança, automatização de SaC (Security as Code) no Pipeline
- Define ainda os controles técnicos e organizacionais adotados, melhores práticas e codificação segura

CULTURA

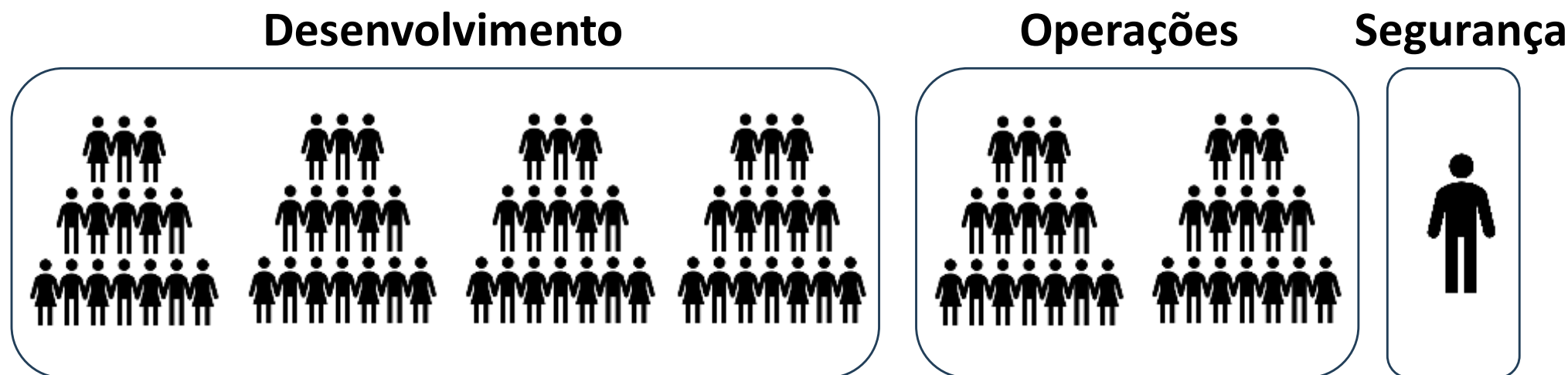
- A cultura é definida como a **interação entre pessoas e grupos** em uma organização
- Ajuda a estabelecer a comunicação e melhorar a compreensão mútua de metas, objetivos e responsabilidades entre as equipes
- Busca **quebrar silos** nas áreas de **Desenvolvimento, Quality Assurance (QA), Operações e Segurança**

CULTURA



- **Governança:** criar métricas abrangentes para garantir que políticas e melhores práticas sejam seguidas. Medir o desempenho de pessoas, processos e tecnologias
- **Processo:** processos de segurança devem ser integrados e devem possuir estreita associação com os processos de desenvolvimento e operações
- **Tecnologia:** adotar automação, SaC, IaC, e gerenciamento de configuração. Tecnologias devem integrar o pipeline de CI/CD
- **Pessoas:** equipes devem comunicar e colaborar entre si para garantir feedback oportuno

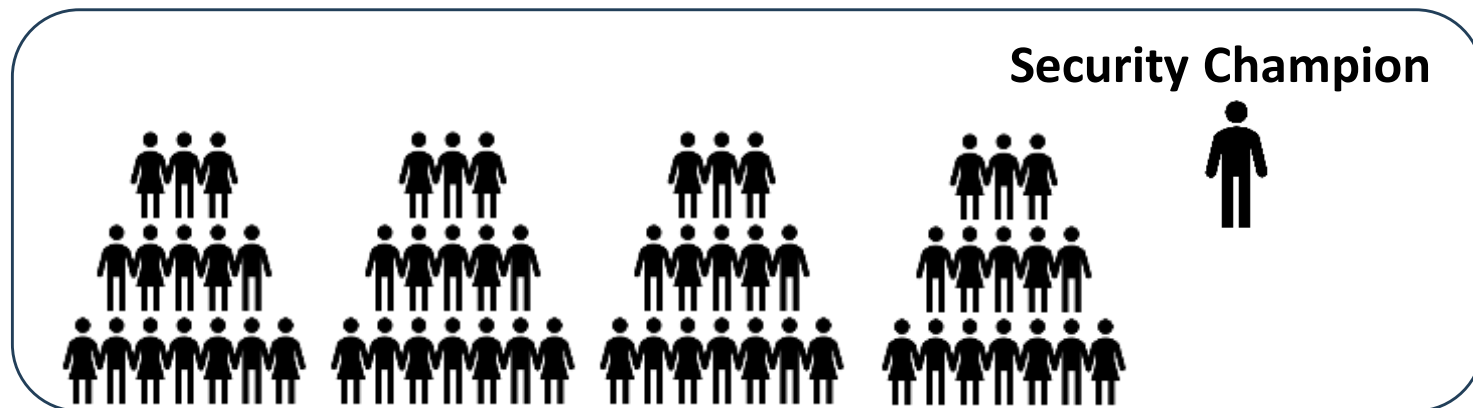
CULTURA: LACUNA DE PROFISSIONAIS DE SI



Cenário típico (**geralmente 100 desenvolvedores, 20 profissionais de Operações e 1 de segurança**) nas empresas quanto à organização das pessoas e envolvidos no contexto de DevSecOps. Este cenário conduz a um ínfimo envolvimento da segurança com as áreas de desenvolvimento e operações

CULTURA: SECURITY CHAMPION

Desenvolvimento



Operações



Segurança



Deve haver **pessoas dedicadas de segurança da informação** para **suportar o processo de segurança no SDLC**. Uma estratégia de um **Programa de Security Champion** pode ser adotada **treinando defensores de segurança na equipe de desenvolvimento** para garantir que a **segurança seja o objetivo principal** em todo o processo de entrega de software. O **Security Champion** deve assumir a responsabilidade de gerenciar (coordenar, rastrear e relatar) as questões de segurança no projeto/produto e fornecer orientação para tomar as decisões necessárias para envolver uma equipe de segurança

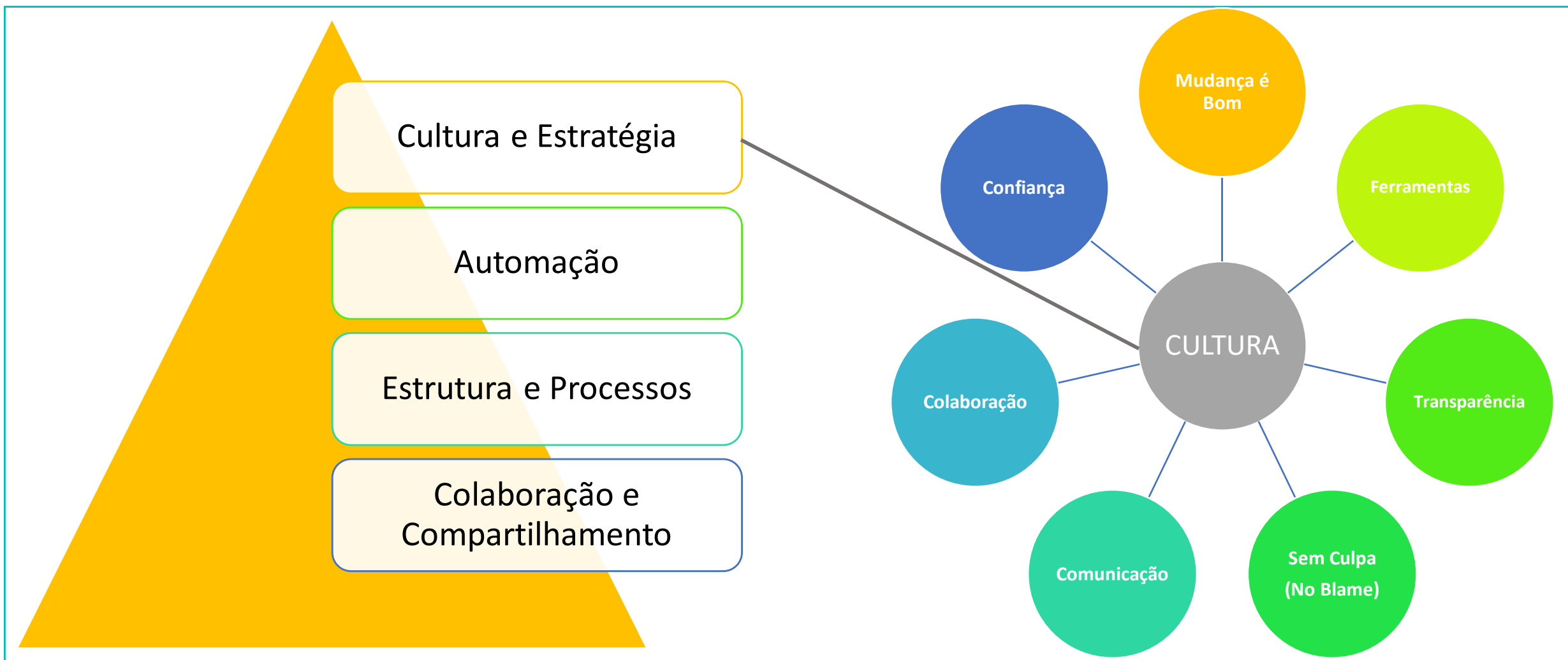
CULTURA: SECURITY CHAMPION

- Aplicar a segurança continuamente no SDLC
- Trabalhar com a área de segurança da informação em estratégias de mitigação de riscos
- Auxiliar as equipes de QA e Testes
- Acompanhar e monitorar testes de software e de segurança
- Auxiliar no desenvolvimento de um ambiente de Integração Contínua (CI)
- Ter conhecimento dos principais ataques cibernéticos contra as aplicações
- Ter conhecimento sobre o OWASP TOP 10, OWASP Testing Guide, Codificação Segura
- Colaborar com as equipes de desenvolvimento e operações para monitorar e registrar problemas de segurança no SDLC

CULTURA: REFORÇAR E RECOMPENSAR O SUCESSO

- Recompensar funcionários com bônus, cartões-presente, brindes da empresa, entre outros, para ações, objetivos, metas atendidas
- Reconhecimentos especiais para casos excepcionais, como promoções, podem ajudar a unir e elevar a colaboração com a segurança no SDLC
- Adaptar estratégias de como as recompensas são obtidas de acordo com a cultura da organização (inclusive, para o Security Champion)
- Crie um programa de recompensa que atenda aos objetivos de implementação da segurança no SDLC (DevSecOps)

MATURIDADE E CULTURA



MATURIDADE DEVOPS

MATURIDADE DEVOPS

Stage 1: Initial

- Ambiente sem DevSecOps e as equipes de Desenvolvimento, Segurança e Operações atuam em silos

Stage 2: Managed

- Foco na colaboração entre as equipes de Desenvolvimento, Segurança e Operação para estabelecer agilidade no processo de desenvolvimento e automatização nos processos operacionais

Stage 3: Defined

- Um processo de negócio bem definido e um processo automatizado são adotados no ambiente organizacional

Stage 4: Measured

- A organização concentra-se em entender o processo e os recursos de automação para habilitar processos de Integração Contínua e Entrega Contínua (CI/CD)

Stage 5: Optimized

- A organização concentra-se em melhorar o trabalho em equipe, otimizando os resultados do negócio, obtendo visibilidade dos resultados e valorizando os esforços da equipe

FOCOS DE MATURIDADE DEVOPS

FOCOS DE MATURIDADE DEVOPS

Maturidade da Aplicação

- Determina a segurança no processo de desenvolvimento de código desde a fase de desenvolvimento até a produção
- A organização deve empregar automação para executar a construção, teste, varredura de segurança e monitoramento no pipeline de implantação para detectar o nível de maturidade nos aplicativos

Maturidade de Dados

- Verifica a capacidade das operações de dados em detectar e automatizar alterações de dados
- Garante que todas as funcionalidades atendam ao nível de maturidade de DevOps por dados (DataOps)

Maturidade de Infraestrutura

- A capacidade de simplificar as habilidades de gerenciamento da infraestrutura pertencentes à automação
- Adota funções de automação, simplificação, autoatendimento e monitoramento para alinhamento com o negócio e medir o nível de maturidade do DevOps por infraestrutura

CAMS

CAMS

Culture

- Interação entre pessoas e grupos em uma organização. Ajuda a estabelecer comunicação e melhorar a compreensão mútua de metas, objetivos e responsabilidades

Automation

- Foco na adoção de um pipeline que consiste em Infraestrutura como Código (IaC), Integração Contínua (CI) e Entrega/Implantação Contínua (CD) para gerenciar a cultura e os objetivos organizacionais

Measurement

- Medir o progresso de várias atividades e processos de DevSecOps usando métricas e indicadores-chave de desempenho (KPIs) para garantir a melhoria contínua (requer o nível máximo de maturidade)

Sharing

- Promove a transparência ao compartilhar novas descobertas, melhores práticas e feedback. Ajuda diferentes equipes a trabalharem juntas para atingir o objetivo comum. Foca nos ciclos de feedback e permite a melhoria contínua da organização

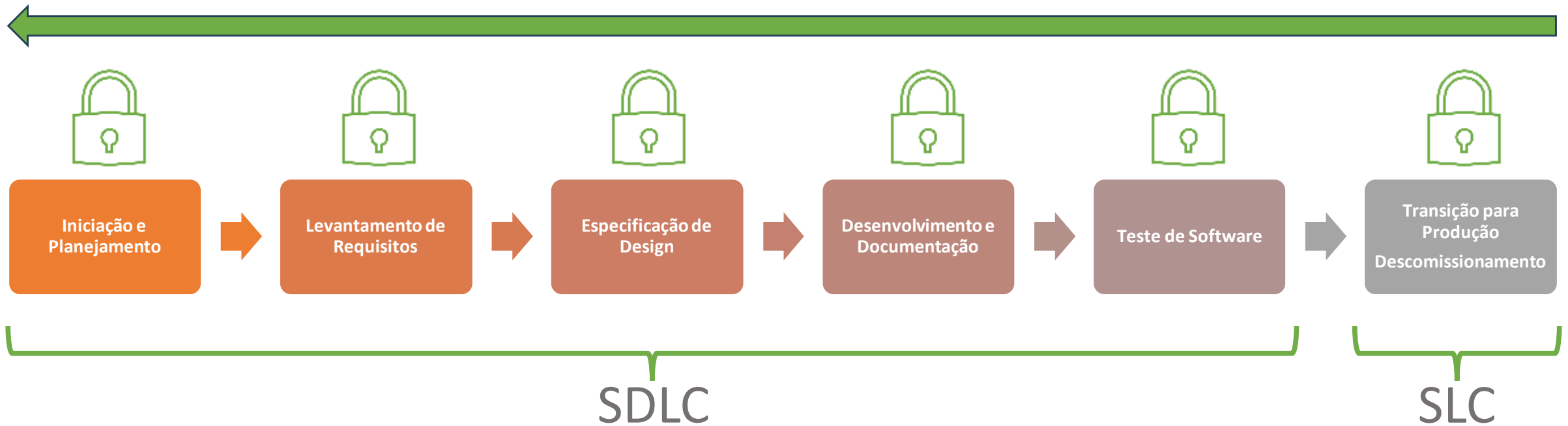
IMPORTANTE: Outras variações incluem CALMS, adicionando o Lean (para **processos enxutos**) e CALMSS, adicionado o Sourcing (para **atividades de terceirização**)

PREMISSAS DA FASE DE PLANEJAMENTO

- Segurança da Informação deve ser introduzida sempre no **início do projeto/produto** no SDLC
- A **cultura de segurança** no SDLC (ou cultura DevSecOps) deve ser **patrocinada pelo Senior Management**
- **Políticas de Segurança** para o **SDLC** e **guias de melhores práticas** devem ser implementadas e adotadas (Controles Organizacionais)
- As **equipes (Dev e Ops)** devem ser **treinadas e conscientizadas**
- Redução do **Débito Técnico de Segurança** (Security Champion pode apoiar)

FASES TÍPICAS DO SDLC

Security Shift-Left (Security SDLC)



Quais os eventuais impactos se os controles de segurança forem abordados somente nos **últimos estágios do SDLC**?

MELHORES PRÁTICAS: OWASP PROACTIVE CONTROLS

C1: Defina
Requisitos de
Segurança

C2: Utilize
Frameworks e
Bibliotecas de
Segurança

C3: Acesso Seguro
à Base de Dados

C4: Codifique e
Sanitize/Escape os
Dados

C5: Valide Todas as
Entradas

C6: Implemente
Identidade Digital
Segura

C7: Aplique
Controles de
Acesso

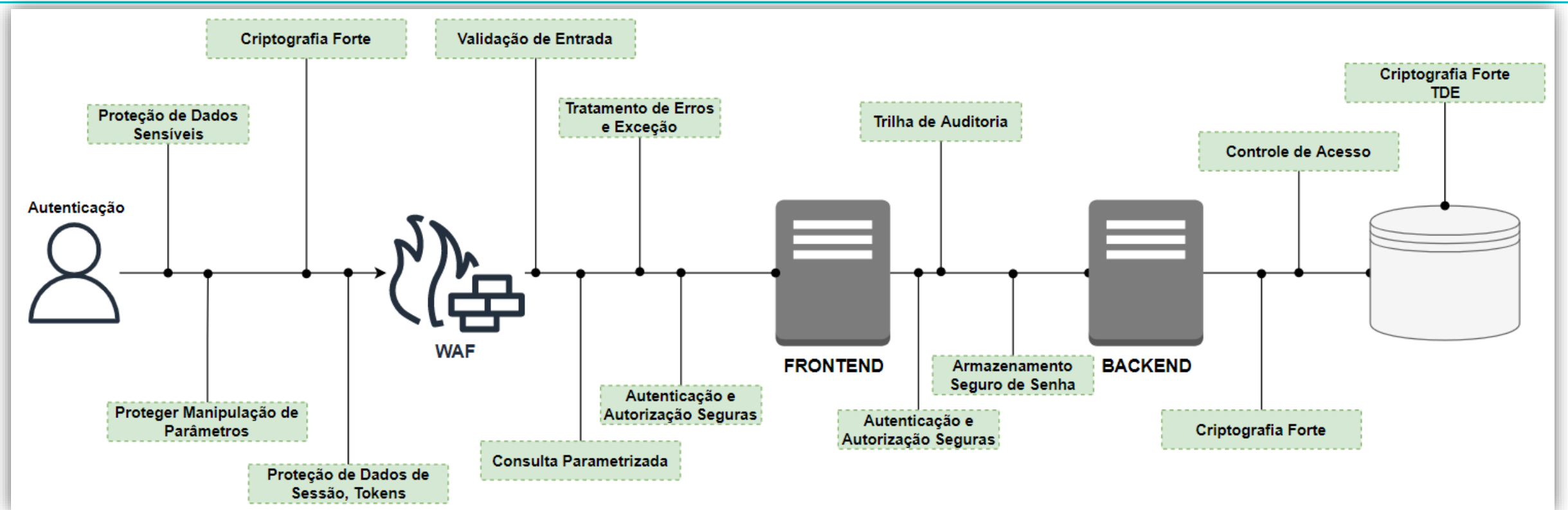
C8: Proteja os
Dados

C9: Implemente
Logs e
Monitoramento de
Segurança

C10:
Faça o Tratamento
de Erros e Exceções

Fonte: <https://owasp.org/www-project-proactive-controls/>

VISÃO GERAL DOS REQUISITOS PRIMÁRIOS DO PROJETO



Um design da aplicação Web é representado definindo os **principais requisitos** para o produto de software a partir de políticas de segurança no **início do projeto do produto**

MODELO SD3+C

MODELO SD3+C

Secure by Design

- Focado em prevenir violações de segurança no SDLC, gerenciamento de acesso, economia de mecanismo (nada desnecessário deve ser implementado), reduzir a superfície de ataque

Secure by Default

- Requisitos de segurança devem ser implantados como parte integrante do produto, balanceando entre requisitos e produtividade, falha segura, controle de privilégios, criptografia

Secure by Deployment

- Padrões de codificação segura, defesa em profundidade, autenticação, auditoria, criptografia de dados em repouso

Communication

- Comunicação segura entre todos e quaisquer componentes para manter a confidencialidade e integridade das informações
- Estabelecer relações de confiança entre componentes

SDL – MICROSOFT SECURITY DEVELOPMENT LIFECYCLE



Fornecer treinamento

Certifique-se de que todos entendam as práticas recomendadas de segurança.



Definir requisitos de segurança

Atualize continuamente os requisitos de segurança para refletir as mudanças na funcionalidade e no cenário regulatório e de ameaças.



Definir métricas e relatórios de conformidade

Identifique os níveis mínimos aceitáveis de qualidade de segurança e como as equipes de engenharia serão responsabilizadas.



Realize a modelagem de ameaças

Use a modelagem de ameaças para identificar vulnerabilidades de segurança, determinar riscos e identificar mitigações.

Fonte: <https://www.microsoft.com/en-us/securityengineering/sdl/practices>

SDL – MICROSOFT SECURITY DEVELOPMENT LIFECYCLE



Estabeleça Requisitos de Projeto

Defina os recursos de segurança padrão que todos os engenheiros devem usar.



Definir e usar padrões de criptografia

Certifique-se de que as soluções criptográficas corretas sejam usadas para proteger os dados.



Gerenciar o risco de segurança do uso de componentes de terceiros

Mantenha um inventário de componentes de terceiros e crie um plano para avaliar as vulnerabilidades relacionadas.



Use ferramentas aprovadas

Defina e publique uma lista de ferramentas aprovadas e suas verificações de segurança associadas.

Fonte: <https://www.microsoft.com/en-us/securityengineering/sdl/practices>

SDL – MICROSOFT SECURITY DEVELOPMENT LIFECYCLE



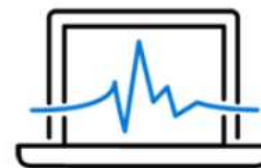
Realizar testes de segurança de análise estática (SAST)

Analise o código-fonte antes de compilar para validar o uso de políticas de codificação seguras.



Realizar testes de segurança de análise dinâmica (DAST)

Execute a verificação em tempo de execução do software totalmente compilado para testar a segurança do código totalmente integrado e em execução.



Realizar testes de penetração

Descubra possíveis vulnerabilidades resultantes de erros de codificação, falhas de configuração do sistema ou outros pontos fracos de implantação operacional.



Estabeleça um processo padrão de resposta a incidentes

Prepare um Plano de Resposta a Incidentes para lidar com novas ameaças que possam surgir ao longo do tempo.

Fonte: <https://www.microsoft.com/en-us/securityengineering/sdl/practices>

GESTÃO DE MUDANÇAS NO MUNDO ÁGIL

Gestão de Mudanças Tradicional

Mudança no Software

A equipe recebe instruções para realizar uma mudança no software, sendo o risco, compreendido como **Alto**.

GMUD

O CAB (Change Advisory Board) irá analisar a mudança e os riscos de segurança associados. O responsável pela mudança é um indivíduo ou **dono da mudança**.

Redução de Risco

O CAB, enquanto órgão consultivo, apresenta os argumentos que requerem ser trabalhados para reduzir o risco ou convencimento para não realizar a mudança

GESTÃO DE MUDANÇAS NO MUNDO ÁGIL

Gestão de Mudanças DevOps

Mudança no Software

A equipe recebe instruções para realizar uma mudança no software, sendo o risco, compreendido como Alto.

CI/CD
Testes Automatizados

O CAB (Change Advisory Board) irá analisar a mudança e os riscos de segurança associados. O responsável pela mudança é a **equipe**.

Redução de Risco

O CAB, enquanto órgão consultivo, apresenta os argumentos que requerem ser trabalhados para reduzir o risco ou convencimento para não realizar a mudança

GESTÃO DE MUDANÇAS NO MUNDO ÁGIL

Categoria	Risco	Programação da Mudança	Requer Aprovação?	Complexidade/Impacto
Padrão	Baixo	Processo e Automação estabelecida	Não	Baixo
Normal	Alto	Programada	CAB	Variável
Urgente	Muito Alto	Imediata	CAB Emergencial	Variável

Para mudanças de baixo risco deve-se considerar a automação do processo de aprovação para tornar o processo de mudanças alinhando com as entregas. As equipes de desenvolvimento e os devidos líderes responsáveis devem analisar o risco para decidir qual fluxo a mudança deve seguir (Padrão, Normal, Urgente).

IMPORTANTE: A maturidade nos testes de Segurança e do software são fundamentais para reduzir os riscos antes de decidir o fluxo da mudança

GESTÃO DE MUDANÇAS ÁGIL

Automação de Processos

- Buscar automatizar os processos, garantindo que os requisitos de conformidade e regulamentação sejam atendidos, minimizando intervenções e erro humano

Adotar CAB

- Use um Change Advisory Board (CAB) que forneça suporte ao gerenciamento de mudanças aprovando solicitações, avaliando e priorizando mudanças, analisando os devidos riscos

Integração com Ferramentas

- Integrar o gerenciamento de mudanças com ferramentas e processos existentes, por exemplo, Jira/ServiceNow/Confluence/Outros para enviar e-mails para aprovações quando uma alteração for enviada ao código pelo desenvolvedor. Requer trilhas de auditoria e lastro de aprovação

Integração Contínua

- Alcançar a Integração Contínua (IC) usando monitoramento e mitigando problemas internos e aqueles relatados por clientes. A estratégia de gerenciamento de mudanças deve ser flexível, robusta em sua abordagem, mas sem complexidades de implementação

Monitoramento e Investigação

- Tente investigar o fluxo de trabalho sempre que um novo código ou uma atualização for introduzida. Deve-se Implementar métricas de desempenho adequadas para calcular os riscos e vulnerabilidades envolvidos nas mudanças

MELHORES PRÁTICAS PARA UM PLANEJAMENTO DE SUCESSO

- Patrocínio do **Senior Management**
- **Quebrar silos e focar na colaboração** (inclui papéis e responsabilidades)
- **Foco no cliente** (qualidade do produto)
- Sempre adotar **metas realistas, tangíveis e exequíveis**
- Adotar **avaliações de desempenho** de equipes e indivíduos
- **Gerenciamento de mudanças** sem impactar a produtividade
- **Segurança não pode ser um gargalo** para a entrega do produto (**automação**)

MELHORES PRÁTICAS PARA UM PLANEJAMENTO DE SUCESSO

- **Monitoramento contínuo** do ambiente de produção e feedback
- **Colaboração ativa** entre as partes interessadas
- **Automatizar testes de software e segurança**
- **Automatize e adote a Integração Contínua (CI)**
- **Automatize e adote a Entrega/Implantação Contínua (CD)**
- **Adote Infraestrutura como Código (IaC) e Segurança como Código (SaC)**

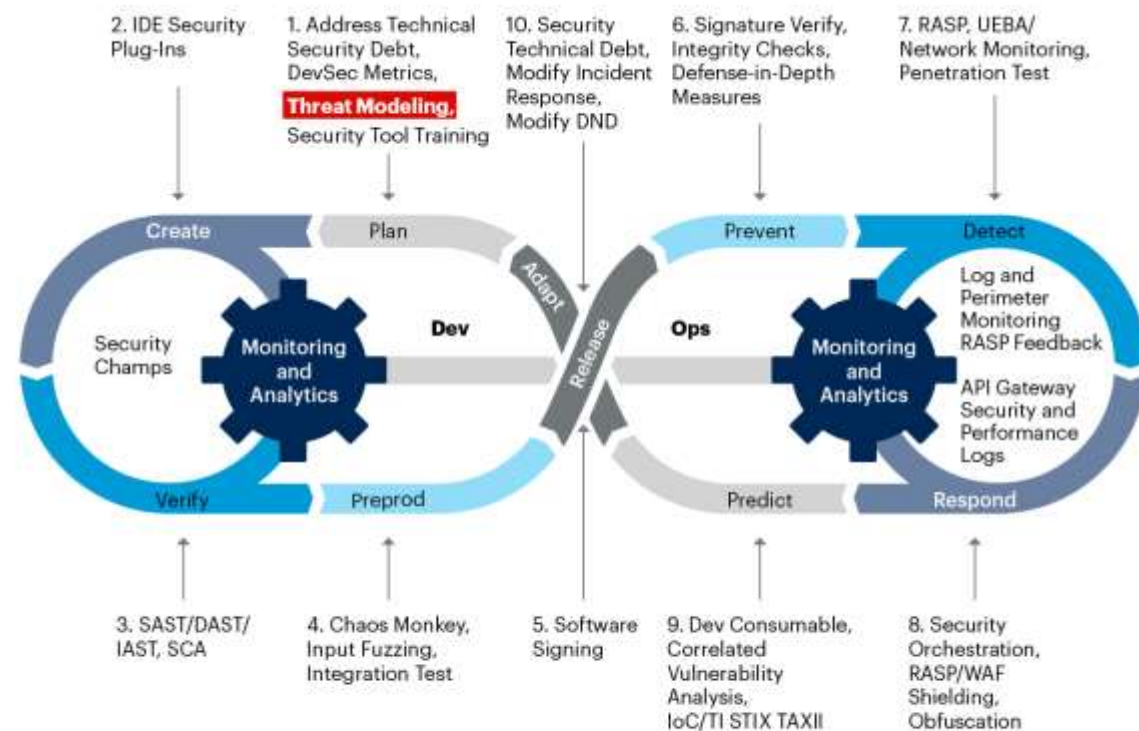
TECNOLOGIAS E PROCESSOS

Adoção de Tecnologias e Processo de Segurança Integradas e Automatizadas no SDLC

- SCM (Source Code Management)
- Análise Estática de Código
- Análise Dinâmica de Código e Pentest
- Análise de Composição de Software
- Análise de Qualidade de Software
- Automatização de Testes
- Monitoramento da Infraestrutura
- Monitoramento da Aplicação (Produto)
- **Centralização de Artefatos de Software**
- Vault (Chave de Criptografia e Segredos)
- **Centralização do Registro de Contêineres**
- Controle de Acesso ao Código Fonte
- Proteção de Branches e Verificação de Pré-Commit
- Política de Revisão de Código (e de Segurança)
- Processo de Controle e Gestão de Mudanças
- **Modelagem de Ameaças**

TECNOLOGIAS E PROCESSOS

The DevSecOps Toolchain



Gartner.
Gartner.



Fonte: <https://www.contrastsecurity.com/security-influencers/gartners-3-crucial-steps-to-inject-security-into-devops>. Gartner **DevSecOps Toolchain** e **Quadrante Mágico**

ESTRATÉGIA EFETIVA DE CULTURA DEVSECOPS

DevSecOps é
Mudança Cultural

Educar a Equipe

Aprendizado
Contínuo dos
Desenvolvedores

Auditorias e
Avaliações de
Segurança

Habilitar Segurança
Baseado em
Código

Automação

DÉBITO



Débito é a distância entre os sistemas atuais e esperados. O sistema esperado é o melhor sistema para atender às necessidades presentes e futuras. O débito leva a uma qualidade de código insatisfatória e representa o trabalho de desenvolvimento extra necessário para remediar a qualidade de código insatisfatória

DÉBITO TÉCNICO E DE SEGURANÇA

Débito Técnico

- Devido ao deadline do projeto, equipes de desenvolvimento buscam atalhos para entregar o aplicativo, resultando em código de baixa qualidade
- Resulta em débito técnico, que é o trabalho extra para resolver os problemas de qualidade
- Muitas tarefas correção de código no backlog

Débito Técnico de Segurança

- Vulnerabilidades e falhas no aplicativo que não foram resolvidas e que podem conduzir uma violação de segurança
- É um subconjunto do Débito Técnico, e pode ser endereçado por meio da adoção da cultura DevSecOps
- Muitas tarefas de remediação de vulnerabilidades no backlog

TRATANDO O DÉBITO TÉCNICO DE SEGURANÇA

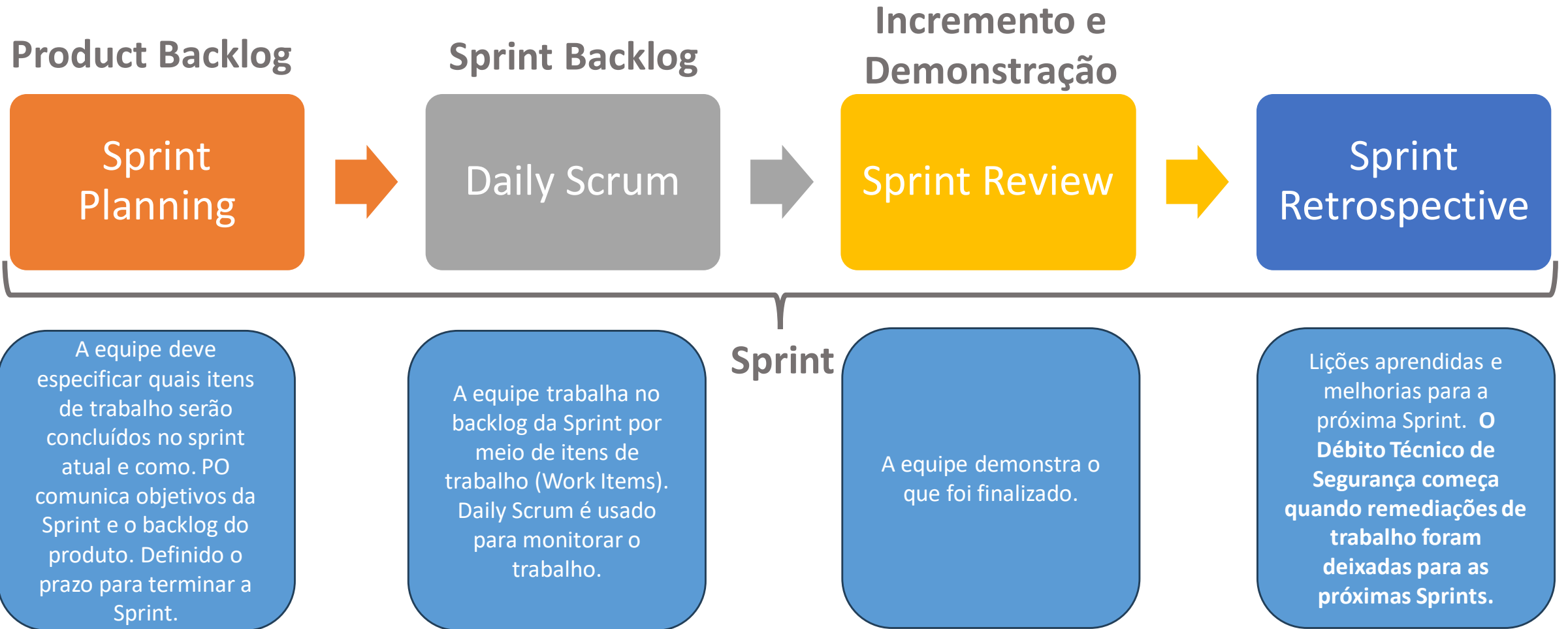
Identificar Pontos de Acesso

- Identificar o uso de anti-padrão na linguagem de programação ou framework adotado
- Avaliar o conjunto de testes existentes
- Garantir que dependências de software estejam atualizadas
- Garantir que funções críticas da aplicação seja avaliada

Débito Técnico de Segurança

- Realizar scan de vulnerabilidade contínuo de forma automatizada, identificando e mitigando as vulnerabilidades no Pipeline
- Remediar as vulnerabilidades tão logo sejam detectadas no início do desenvolvimento por meio de scan contínuo do Pipeline

TRATANDO O DÉBITO TÉCNICO DE SEGURANÇA



TRATANDO O DÉBITO TÉCNICO DE SEGURANÇA

- O Product Owner (PO) e equipe de desenvolvimento devem ser conscientizados e apresentar-lhes suas responsabilidades para evitar débito técnico de segurança
- Ter em mente o princípio neste contexto que também é aplicável:
“**responder a mudanças** mais que seguir um plano”
- Vulnerabilidades devem ser consideradas como bugs e falhas que devem ser tratados antes de se tornarem débito técnico de segurança

TRATANDO O DÉBITO TÉCNICO DE SEGURANÇA

- Hooks de Pré-Commit devem ser configurado nas estações de trabalho de todos os desenvolvedores e ambiente de desenvolvimento
- Usuários válidos, e-mails autorizados e configurações de segurança são definidos para o SCM (Source Code Management) nas estações de trabalho dos desenvolvedores e ambientes de desenvolvimento
- Hooks de Pré-Commit garantem a integridade dos dados do usuário, push de informações sensíveis acidentais, entre outros

TREINAMENTO E CONSCIENTIZAÇÃO

Adotar Métodos Efetivos de Treinamento

Práticas de
Codificação
Segura

OWASP TOP 10

Princípios de
Segurança da
Informação

Exploração de
Vulnerabilidades

Modelagem de
Ameaças

Avaliação da
Eficácia do
Treinamento



PUC Minas

DESAFIO

DESAFIO

Uma empresa do setor de tecnologia da informação está com uma frente de inovação em rápida ascensão. As equipes estão animadas pois deploys contínuos tem permitido atender os objetivos da empresa. Contudo, após quatro meses de empenho, observou-se que apesar do número de deploys ser um fator positivo, o número de rollback também aumentou consideravelmente devido a impactos no ambiente de produção e estabilidade da plataforma. A empresa não possuía um processo de gestão de mudanças e resolveram adotar um processo baseado no ITIL. O resultado foi positivo, diminuindo impactos no ambiente, contudo, como trade-off, o desenvolvimento ficou engessado, culminando em atrasos na entrega de features do produto, equipes descontentes e aumento de aspectos burocráticos. A empresa precisa resolver este problema equilibrando agilidade e segurança no processo.

DESAFIO

Quais as suas considerações para ajudar a empresa de tecnologia da informação para alcançar seus objetivos e o devido equilíbrio esperado?



PUC Minas



PUC Minas

PLANEJANDO A MODELAGEM DE AMEAÇAS DO PRODUTO

SCRUM: ARTEFATOS E EVENTOS

História do Usuário

- Requisitos dos usuários. São documentos que descrevem os requisitos do sistema, contudo, histórias são documentos resumidos, com apenas duas ou três sentenças, que define o que se deseja que o sistema faça
-

Backlog do Produto

- Compreende uma lista de histórias, ordenada por prioridades, pelo Dono do Produto e constituem uma descrição resumida das funcionalidades que devem ser implementadas no produto

Sprint

- Em Scrum trata-se de uma iteração. Scrum é um método iterativo, no qual o desenvolvimento é dividido em sprints, de até um mês

Planejamento do Sprint

- Reunião na qual todo o time se reúne para decidir as histórias que serão implementadas no sprint que será iniciado. Marca o início de um sprint. O Dono do Produto propõe as histórias (a partir de um Épico) e os desenvolvedores validam a velocidade para implementá-las. Os desenvolvedores desmembram as histórias em Tarefas estimando a duração.

Backlog do Sprint

- Artefato gerado ao final do Planejamento do Sprint. É uma lista com as tarefas do sprint e a duração destas. Apesar de determinadas tarefas poderem ser alteradas e a duração, o **objetivo do sprint** (sprint goal) - lista de histórias selecionadas para o sprint – não deve ser alterada

SCRUM: PAPÉIS E RESPONSABILIDADES

Product Manager (PM)	Product Owner (PO)	Desenvolvedores
▪ Analisar o mercado ▪ Definir as estratégias do produto ▪ Avaliar a concorrência ▪ Gerenciar a visão do produto ▪ Traduz a visão estratégica do produto do Senior Management (CEO) para as áreas ▪ Atua frente ao âmbito estratégico da organização ▪ Dissemina o método ágil Scrum na organização	▪ Representa o cliente dentro da organização ▪ Criar a visão do produto ▪ Tomar decisões no produto ▪ Adota metodologias ágeis ▪ Desenvolve e monitora métricas ▪ Responsável pelo Backlog do Produto ▪ Define o roadmap do produto ▪ Refina o Backlog ▪ Conduz a Daily Scrum ▪ Conduz a reunião de revisão do sprint	▪ Definem as tarefas a partir das histórias do usuário ▪ Definem a velocidade possível de implementação ▪ Participam da Daily Scrum (indicando o que foi feito, o que será feito no dia e problemas enfrentados) ▪ Participam da Revisão do Sprint (resultados da sprint, apresenta o produto para o cliente, aprovação das histórias ou reprovação, que volta para o backlog do produto e requer novas sprints)

SCRUM: PAPÉIS E RESPONSABILIDADES



História do Usuário



Product Owner

As equipes geralmente compreendem entre 3 e 9 desenvolvedores



Desenvolvedores



Tarefas e prazos



Daily Scrum



Sprint Review

METODOLOGIAS DE THREAT MODELING

- **STRIDE** (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service (DoS), and Elevation of Privilege)
- **PASTA** (Process for Attack Simulation and Threat Analysis)
- **TRIKE**
- **VAST** (Visual, Agile, and Simple Threat)
- **OCTAVE** (Operationally Critical Threat, Asset, and Vulnerability Evaluation)

PROCESSO DE MODELAGEM DE AMEAÇAS



MEDOLOGIA STRIDE

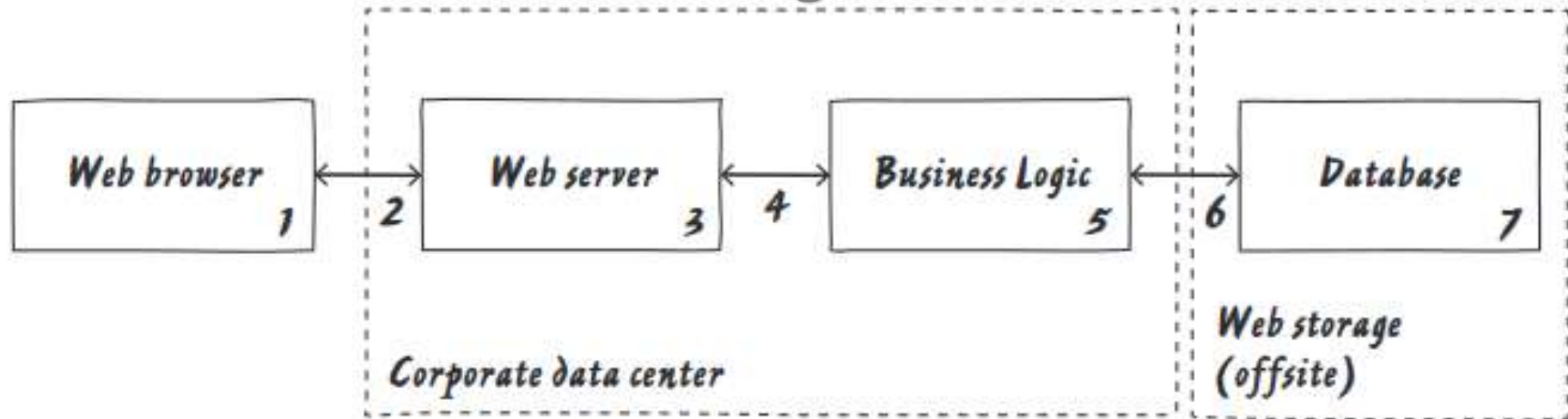
STRIDE é uma metodologia para identificar ameaças à segurança computacional desenvolvido por Praerit Garg e Loren Kohnfelder na Microsoft

	TIPO DE AMEAÇA	VIOLAÇÃO	MEIOS DE VIOLAÇÃO
S	Spoofing	Autenticação	Acessar e usar as credenciais de outro usuário, como nome de usuário e senha
T	Tampering	Integridade	Modificar sem autorização os dados em trânsito, repouso ou em uso
R	Repudiation	Não Repúdio/Autenticidade	Afirmar que não é responsável pela ação
I	Information Disclosure	Confidencialidade	Provê informação para quem não está autorizado
D	Denial of Services	Disponibilidade	Negar ou impedir acesso a recursos
E	Elevation of Privilege	Autorização	Obter acesso privilegiado a recursos para obter acesso não autorizado a informações ou comprometer um sistema.

MEDOLOGIA STRIDE

- Quais são as funções de negócios do sistema?
- Quais são os papéis dos atores que interagem com o sistema?
- Que tipo de dados o sistema processa e armazena?
- Existem requisitos comerciais ou legais que afetam a segurança?
- Com quantos usuários o sistema deve lidar?
- Quais são as consequências se o sistema não fornecer confidencialidade, integridade ou disponibilidade dos dados e serviços que o sistema lida?

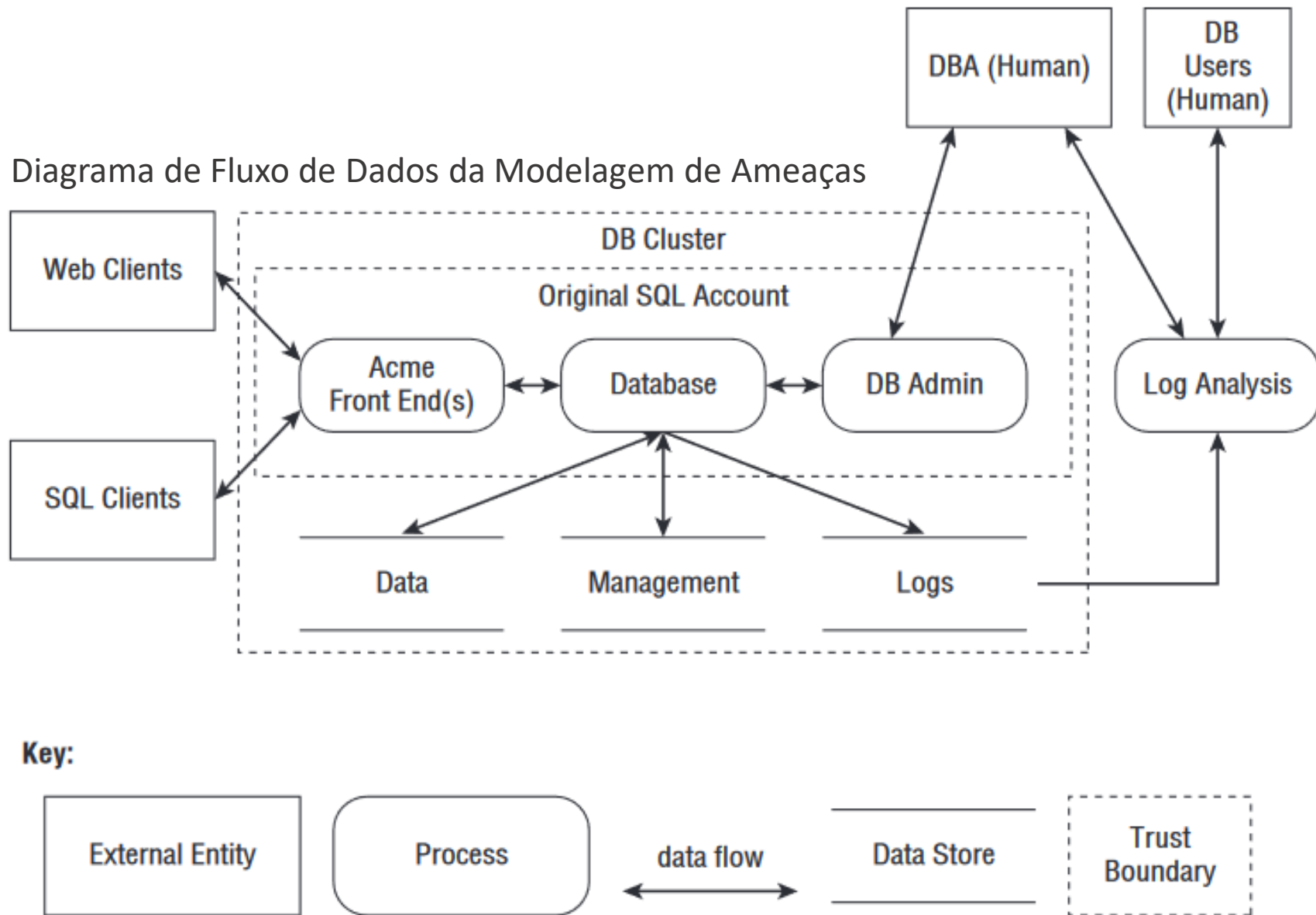
Produto do Negócio



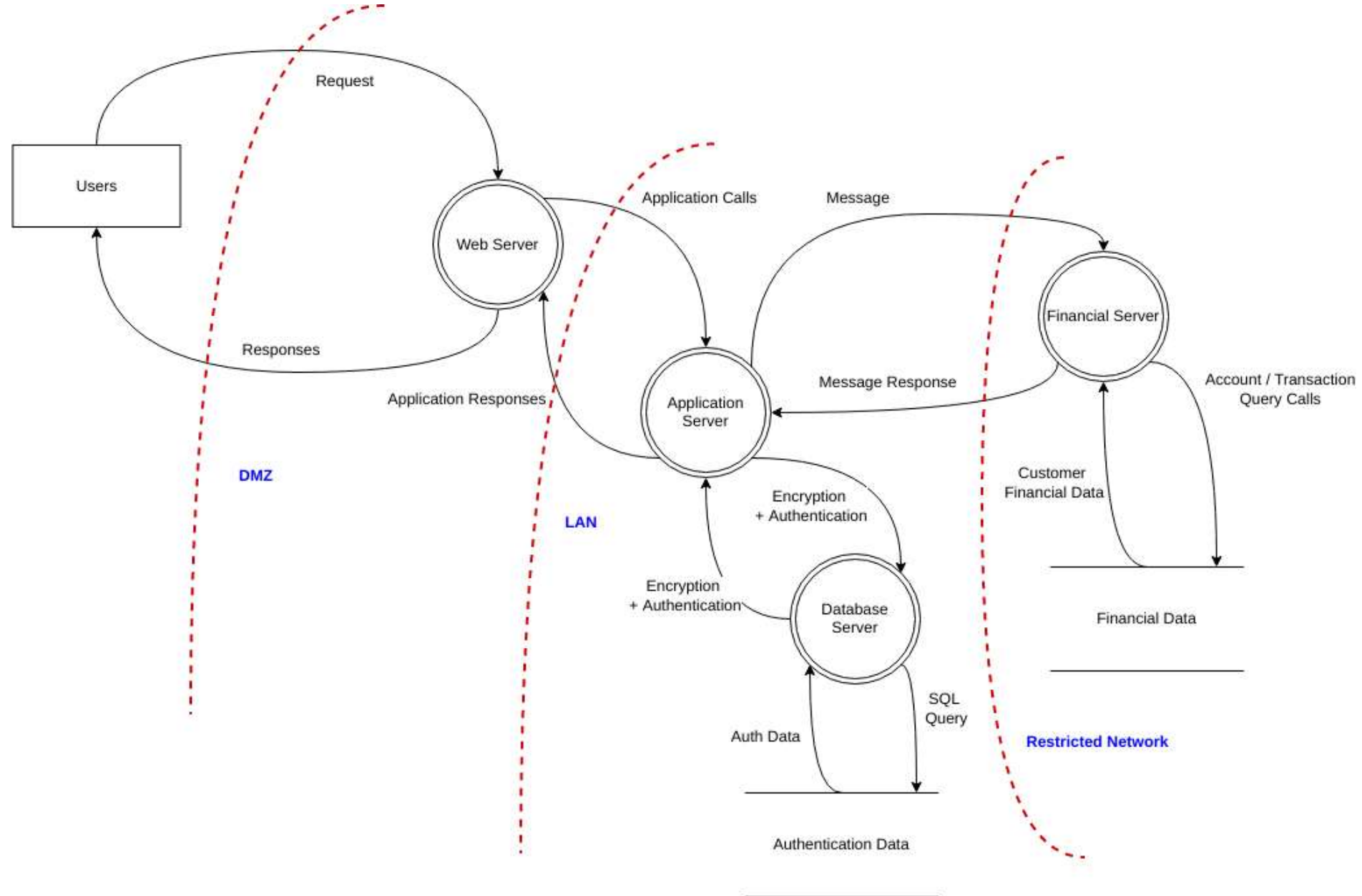
Limites de Confiança e Superfície de Ataque

Referência: Threat Modeling: Designing for Security. SHOSTACK, Adam. John Wiley & Sons, Inc. 2014. EUA.
Limites de confiança e superfície de ataque.

Diagrama de Fluxo de Dados da Modelagem de Ameaças

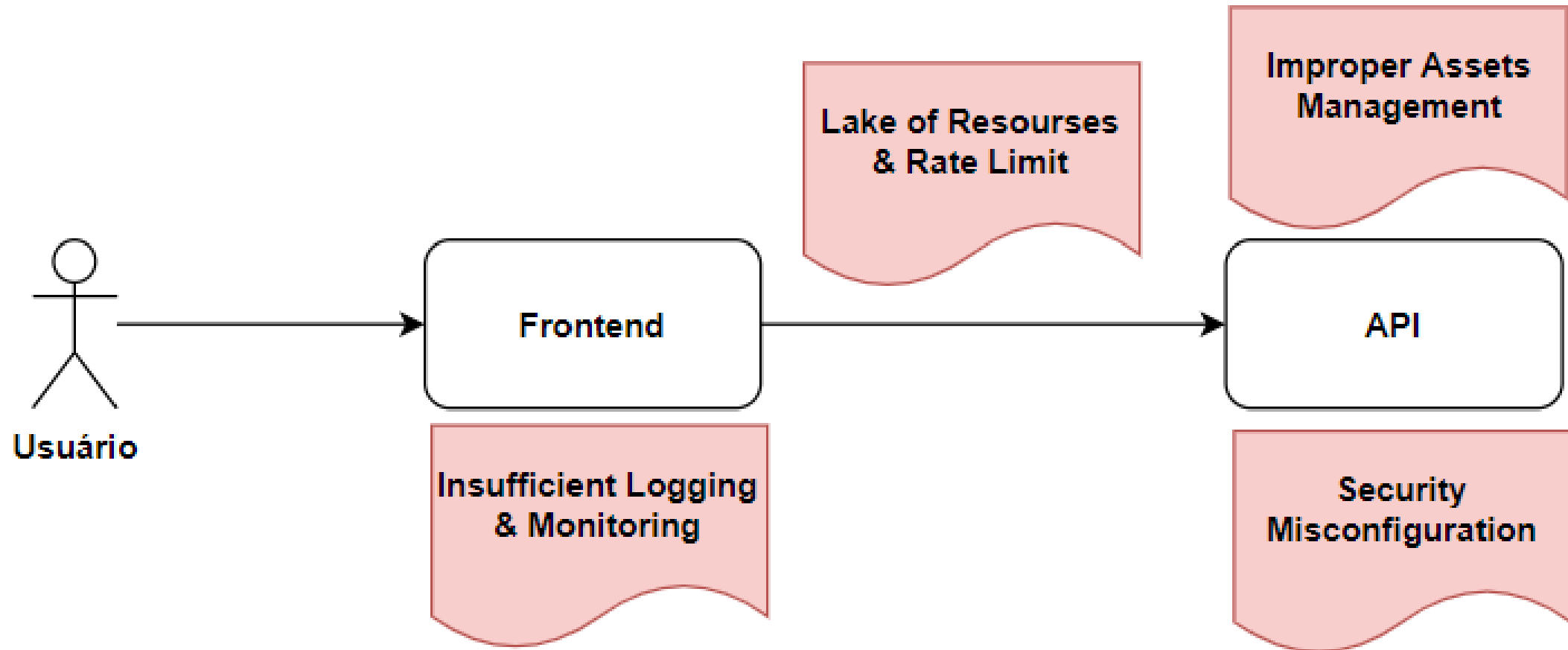


Referência: Threat Modeling: Designing for Security. SHOSTACK, Adam. John Wiley & Sons, Inc. 2014. EUA.
Diagrama de Fluxo de Dados Moderno na Modelagem de Ameaças.

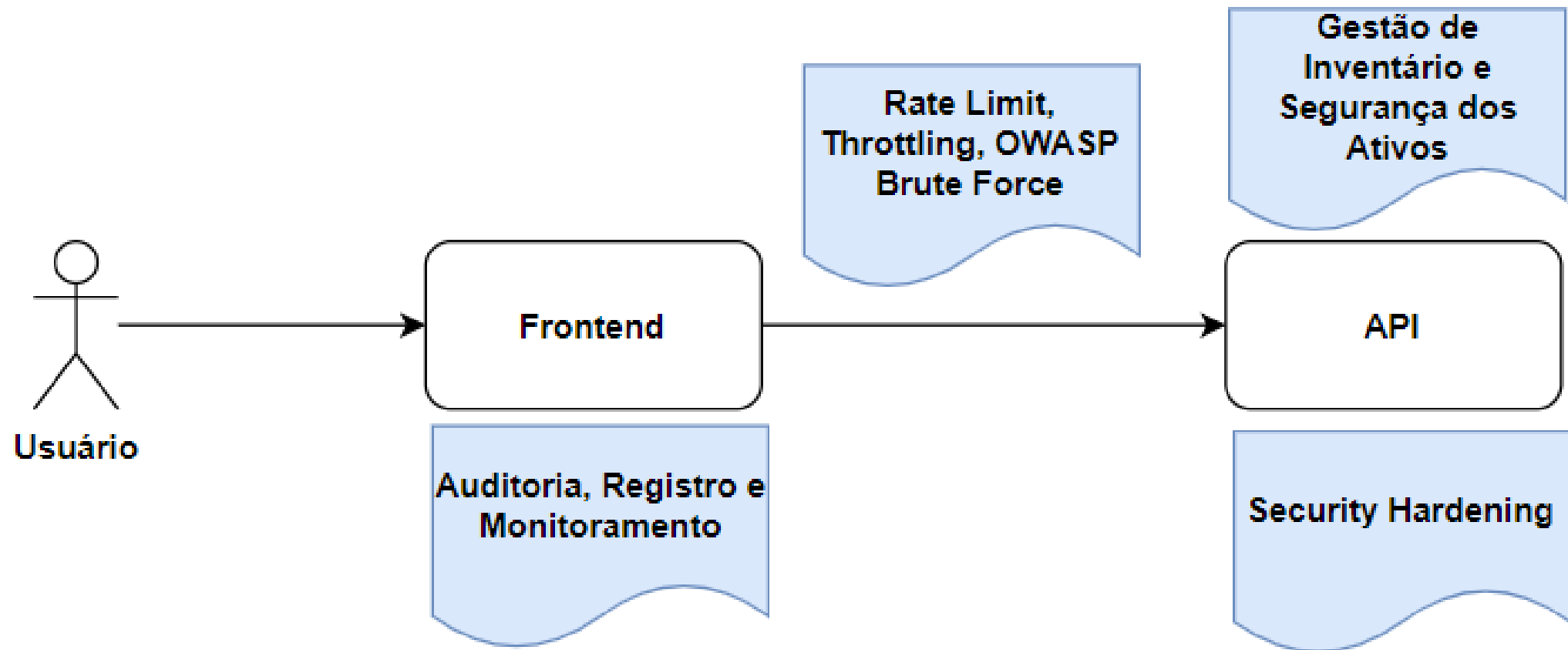


Referência: <https://online.visual-paradigm.com>. Um diagrama de fluxo de dados generalista de modelagem de ameaças para um Web Site abordando os limites de confiança.

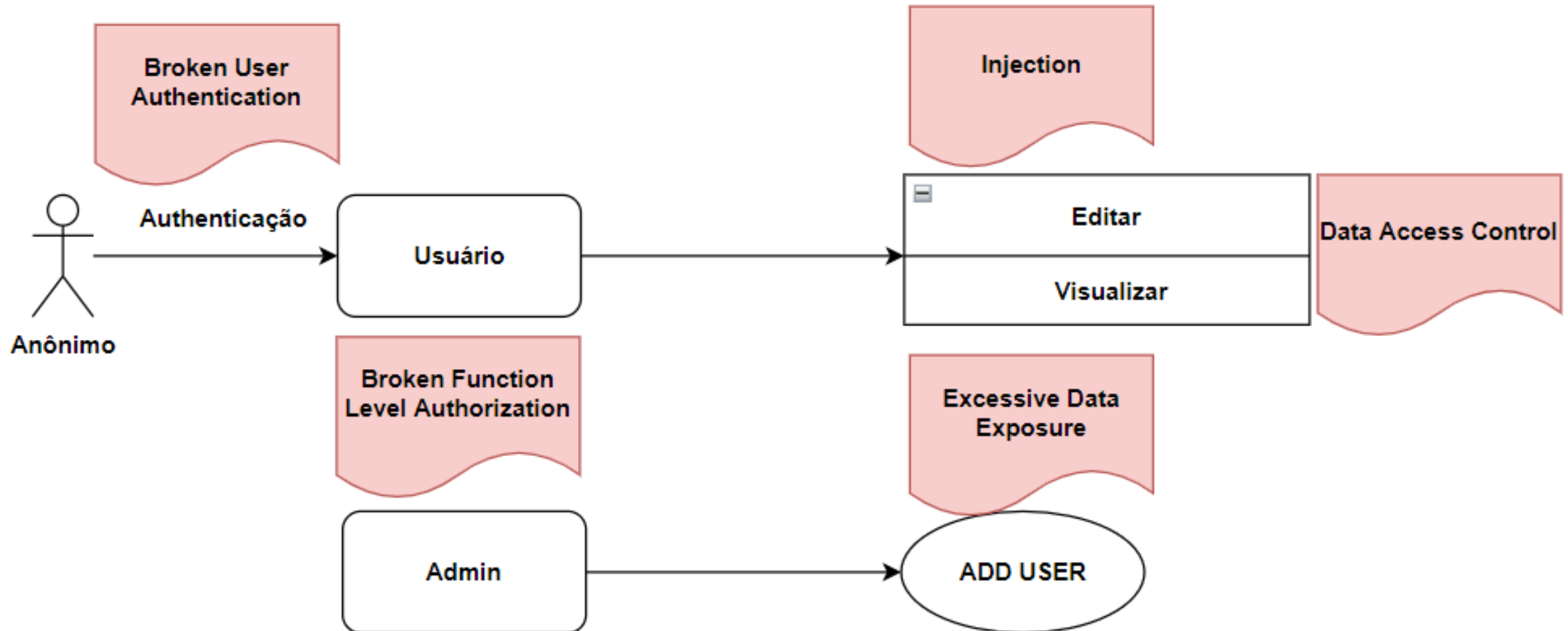
Modelagem de Ameaças em API utilizando o OWASP Top 10 de API como catálogo de ameaças



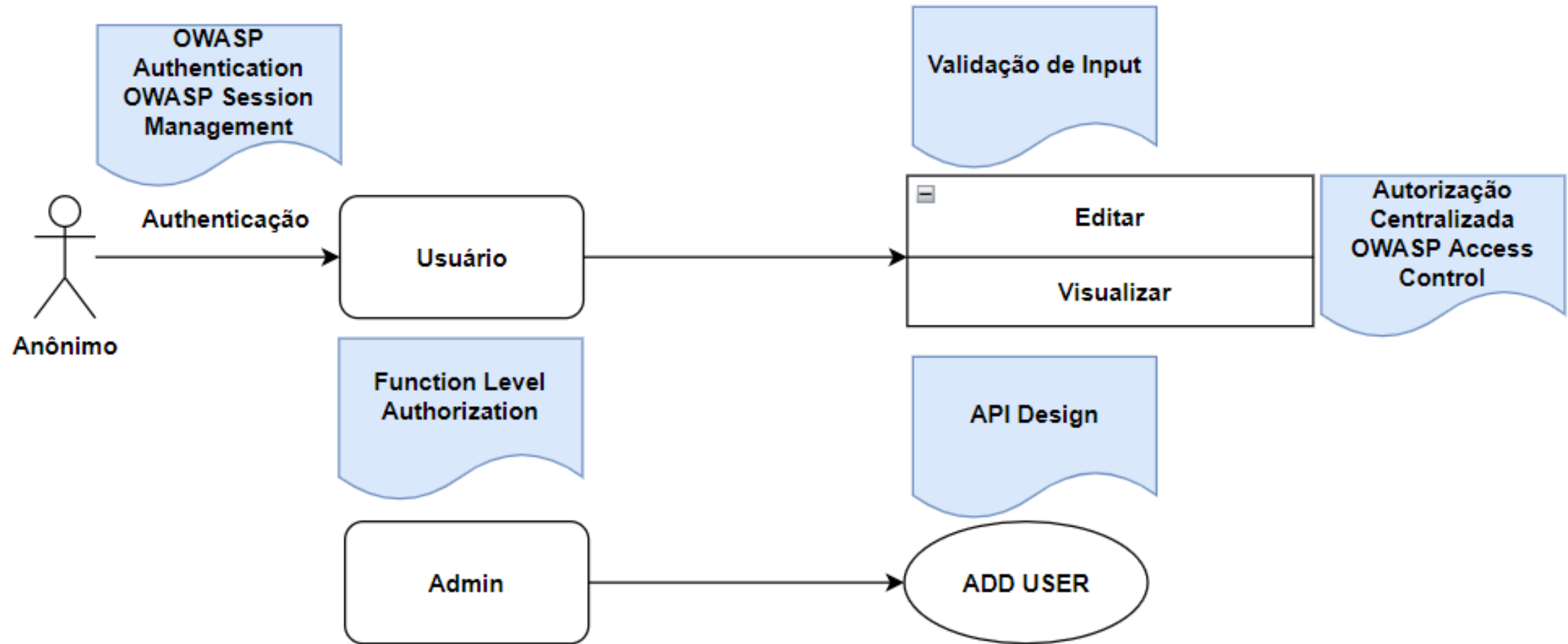
Modelagem de Ameaças em API utilizando o OWASP Top 10 de API e aplicando os devidos controles de segurança



Modelagem de Ameaças em API e Web utilizando o OWASP Top 10 de API e Web como catálogo de ameaças



Modelagem de Ameaças em API e Web utilizando o OWASP Top 10 de API e Web aplicando os devidos controles de segurança



PARA PENSAR... E RESPONDER

- Considerando as fases de um típico SDLC tradicional, como Waterfall em qual fase a Modelagem de Ameaças deve ocorrer?
- Deve-se utilizar Modelagem de Ameaças em métodos ágeis? Em caso positivo, em que fase a modelagem de ameaças deve ocorrer no SCRUM?

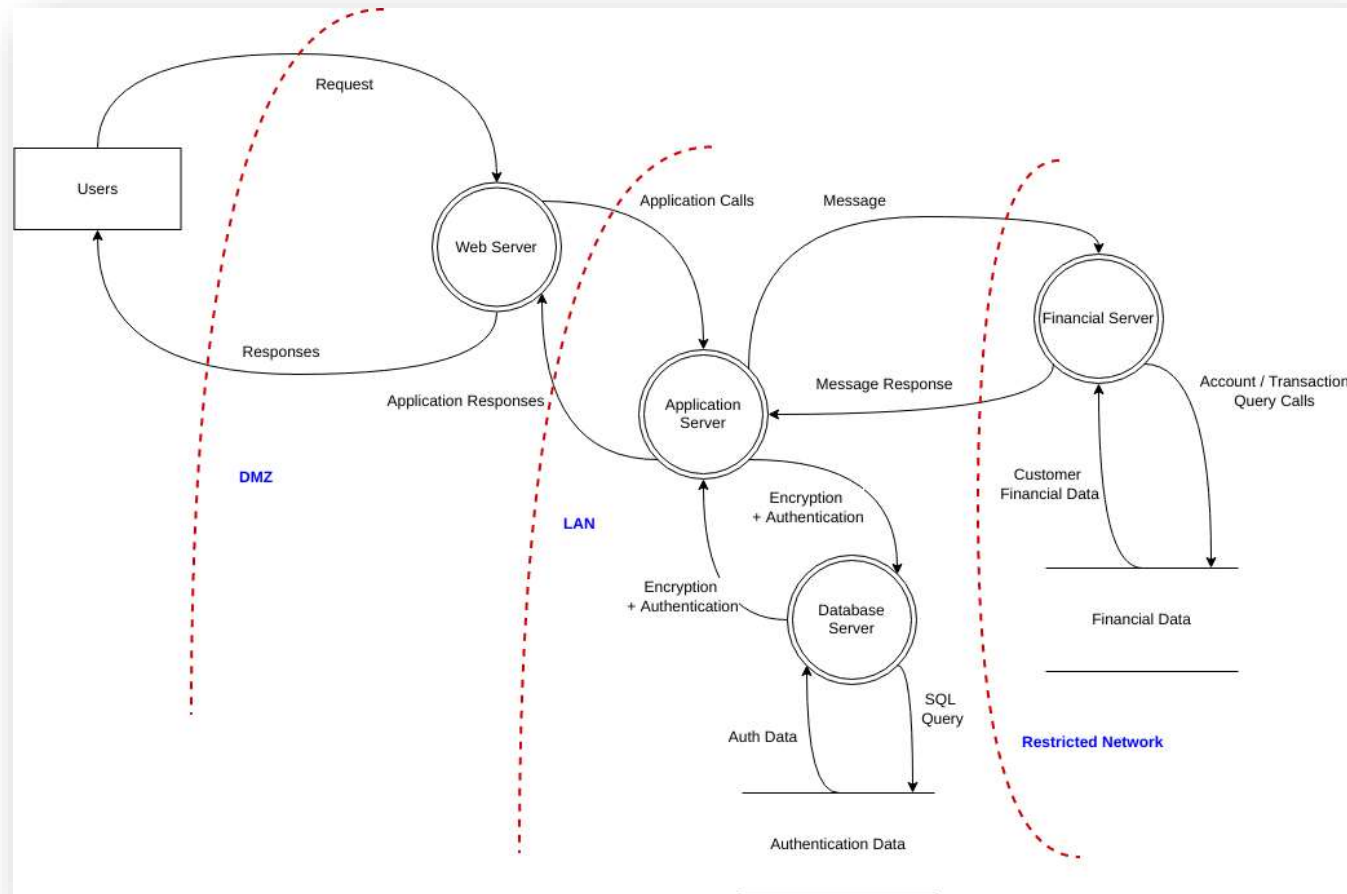


PUC Minas

DESAFIO

DESAFIO

Uma empresa realizou o DFD (Diagrama de Fluxo de Dados) da seguinte Aplicação. Quais as possíveis ameaças?





PUC Minas



PUC Minas

SECURITY SDLC: CODE

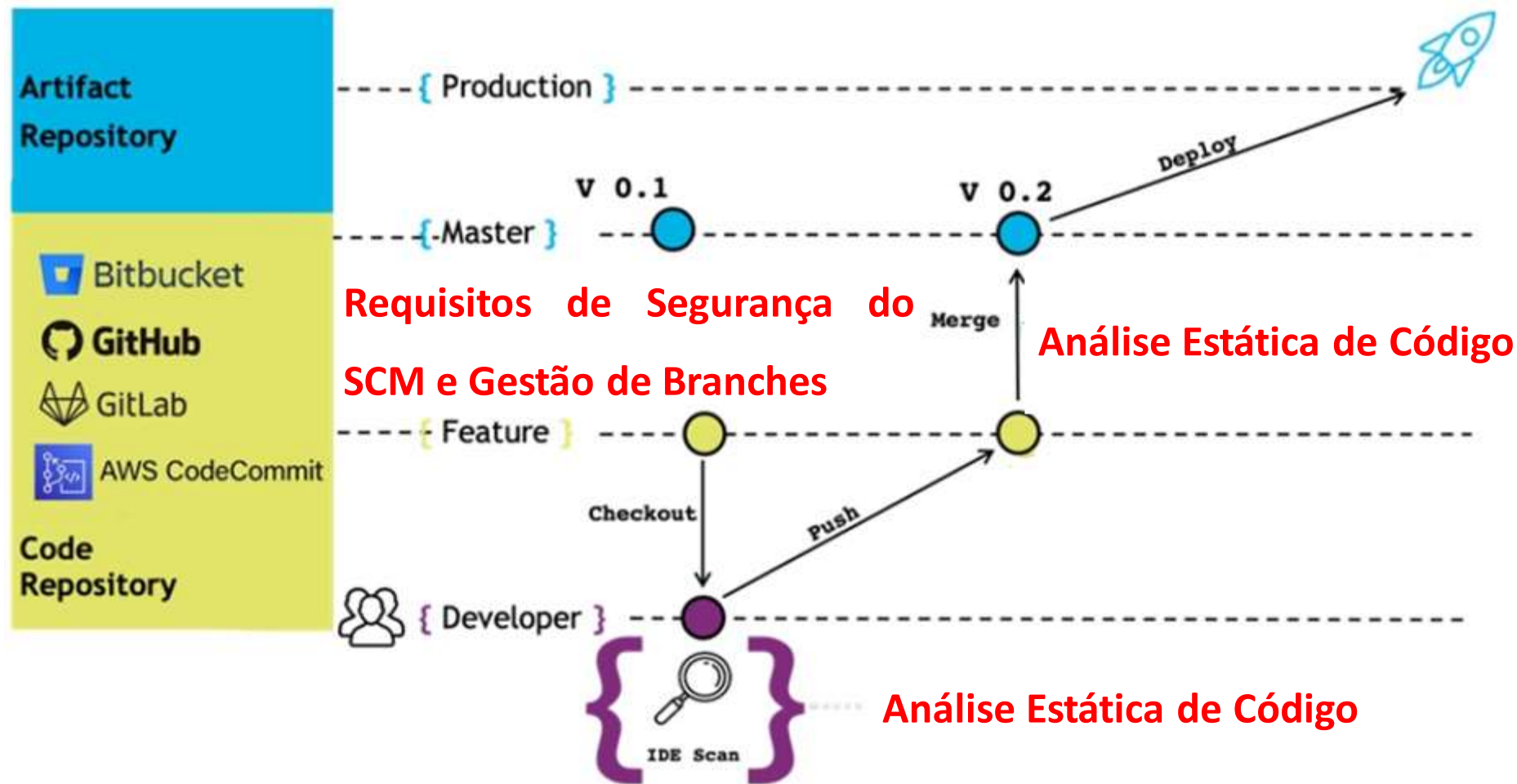
OBJETIVO

Compreender os principais conceitos primários e fundamentais que envolvem a fase de CODE, no SDLC

FASE DE CODE

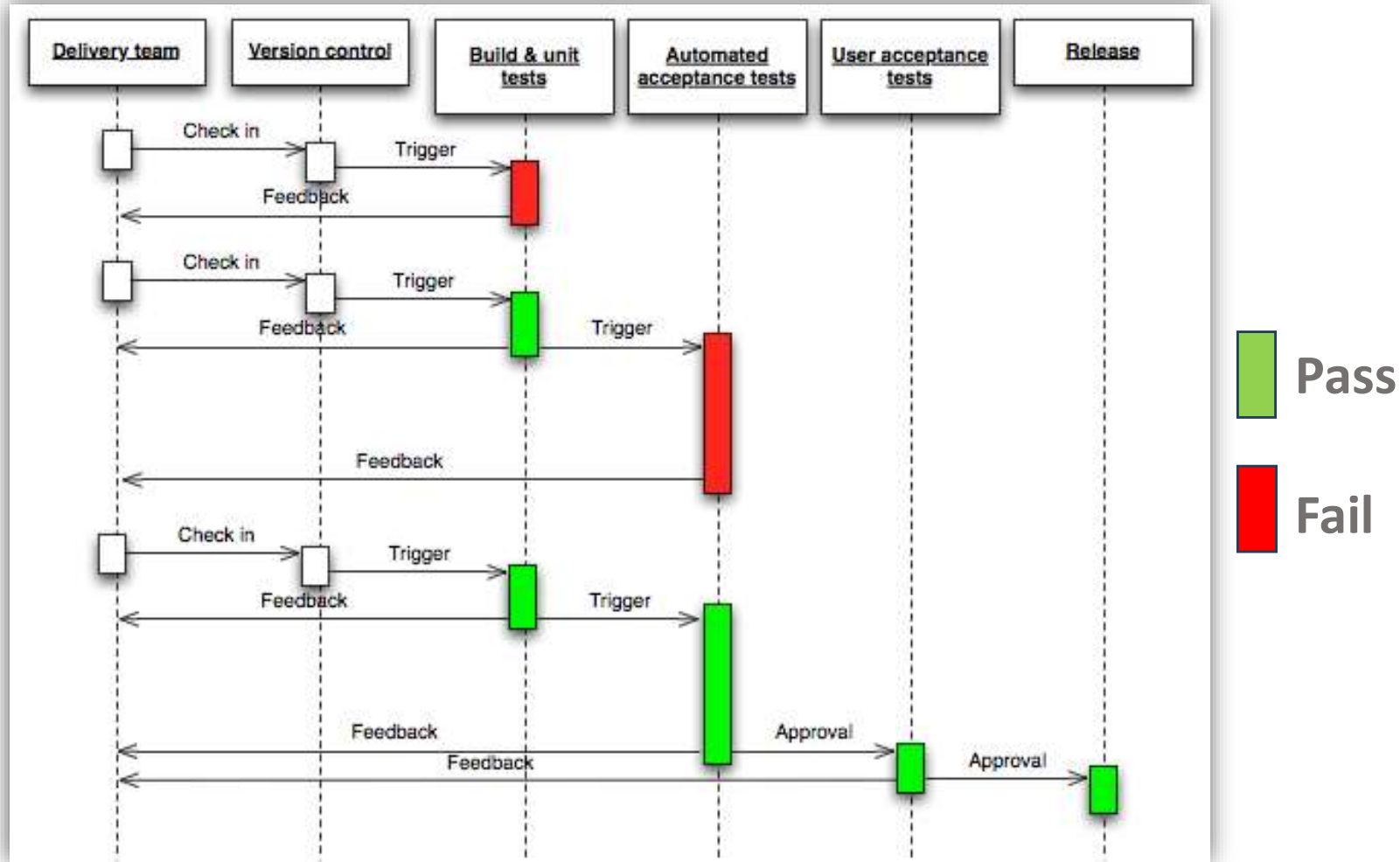
- Esta fase tem como foco a **codificação do produto** pelas equipes competentes
- A **introdução de análise estática de código** é fundamental nesta fase
- A análise estática de código deve ser parte do Pipeline DevOps e ocorrer de forma automatizada
- A análise estática de código deve ser considerada desde a IDE de Desenvolvimento dos colaboradores

OBJETIVOS GERAIS



Fonte: Adaptado de: <https://aws.amazon.com/pt/blogs/apn/embracing-devsecops-building-security-into-cloud-native-development-workflows/>

FALHAS DE SEGURANÇA NO PIPELINE



Fonte: <https://continuousdelivery.com/2010/02/continuous-delivery/>

Dilema

Quando uma solução de análise estática de código encontra uma vulnerabilidade, qual ação tomar?

Quebrar o Pipeline ou Não Quebrar o Pipeline?

Eis a questão!

QUEBRA DO PIPELINE

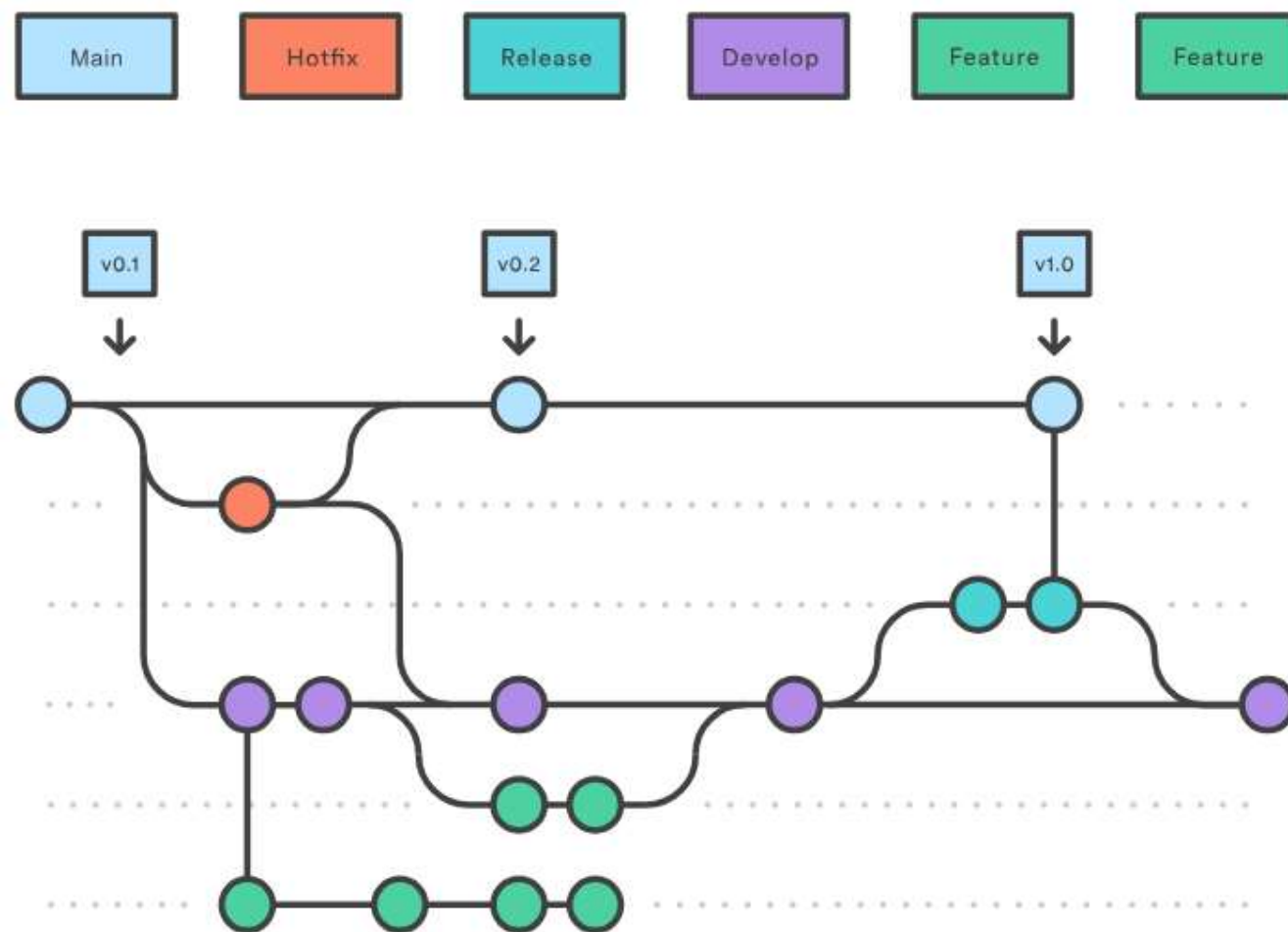
- **O que fazer em caso de detecção de vulnerabilidade? Estratégias:**
 - **Quebrar o Pipeline** em função da gravidade da vulnerabilidade (Requer Maturidade)
 - Adotar **mecanismo de aprovação com responsabilização** sem quebrar o Pipeline (Requer Supervisão, Treinamento e Conscientização Eficaz)

SCM – SOURCE CODE MANAGEMENT

- Repositório onde o **código-fonte** é **centralizado** e **devidamente integrado ao Pipeline**
- Código-fonte majoritariamente é classificado como **informação confidencial** e segredo **industrial**
- Requer rígido controle de acesso e proteção contra exfiltração de dados (DLP, Controle de Acesso, Revisão de Acesso)
- Requer configurações de **proteção de Branch**, estratégia de **versionamento e Branch Workflow**, **revisores** obrigatórios e número de revisores, alertas de **divulgação não autorizada**, identificação de **informações sigilosas**, processo de aprovação, auditoria, entre outros

SCM – SOURCE CODE MANAGEMENT

- Cada organização pode definir o workflow do gerenciamento de Branches (ao lado Gitflow Workflow)
- Idealmente a análise estática de código deve ocorrer em todas as Branches anteriores à Branch Main (qual a razão?)



Fonte: <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>

SCM – SOURCE CODE MANAGEMENT



SCM – SOURCE CODE MANAGEMENT - GAS

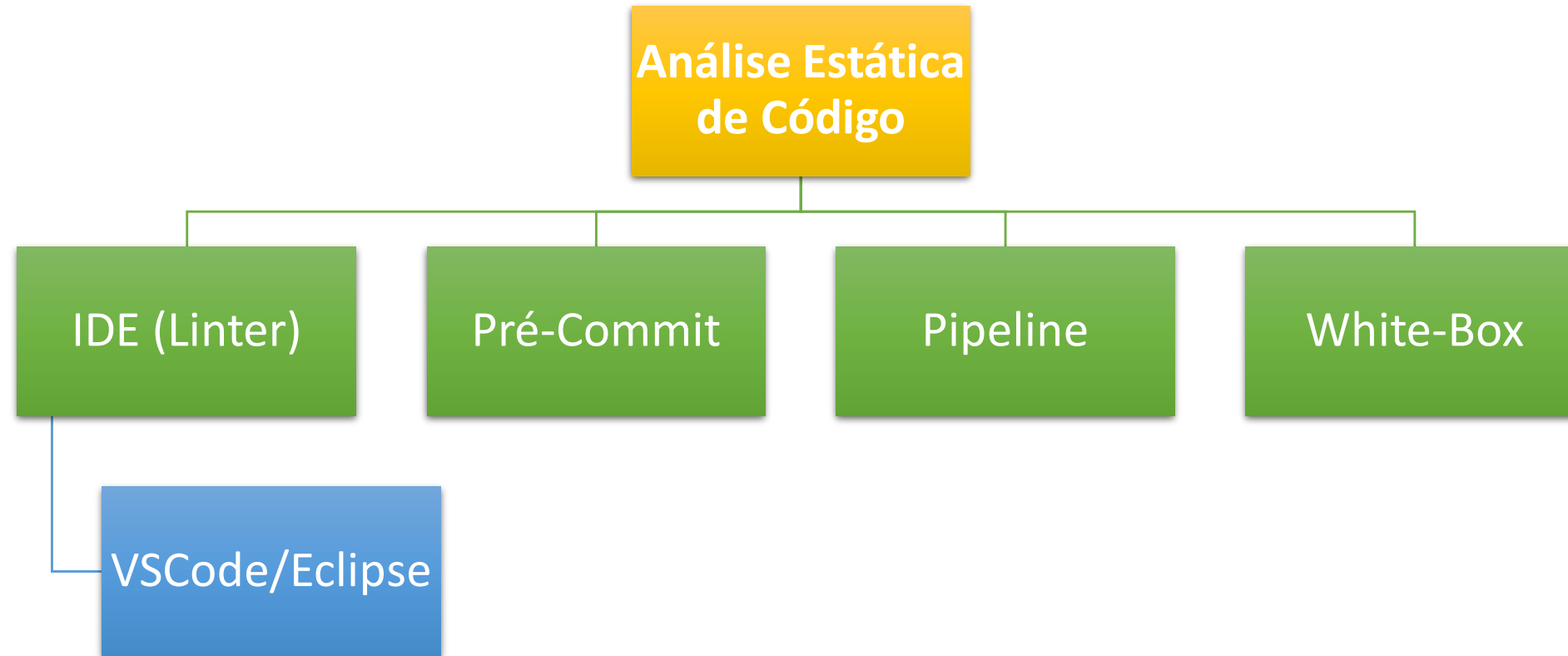
GITHUB ADVANCED SECURITY (GAS)

Code Scanning	Encontra possíveis vulnerabilidades de segurança e erros de codificação usando o CodeQL ou ferramenta de terceiros
CodeQL CLI	Processa localmente projetos de software ou gera resultados de scanning de código para upload para o GitHub
Secrets Scanning	Detecta segredos (chaves e tokens) verificados em repositórios privados. Se a proteção push estiver ativada, também detecta segredos quando eles são enviados para o repositório
Custom Auto-Triage Rules	Gerenciamento de alertas Dependabot. Com regras de triagem automática personalizadas permitindo controle sobre os alertas que deseja ignorar ou acionar uma atualização de segurança
Dependency Review	Apresenta o impacto total das mudanças nas dependências e detalhes de quaisquer versões vulneráveis antes de mesclar o Pull Request

COMMIT E PULL REQUEST

- O **SCM** deve ser **integrado** com **soluções de análise estática de código**
- Forma de notificar a equipe sobre as alterações feitas por um usuário em uma branch específica do repositório
- Um pull request é uma etapa realizada antes de mesclar as alterações com a branch principal e adicionar commits de acompanhamento para discutir as alterações com as partes envolvidas
- A **análise estática de código** deve ocorrer no Pull Request como uma **tarefa obrigatória**

ANÁLISE ESTÁTICA DE CÓDIGO





PUC Minas

DESAFIO

DESAFIO

Uma empresa do setor de tecnologia da informação resolveu adotar a seguinte estratégia para a gestão das Branches:

- Em caso de novas features, os demandantes abrem um chamado para realizar a implementação. Então, uma nova Branch intitulada Feature-NumeroChamado é criada. Ao final, é realizado o merge com a Branch Main.
- Em caso de hotfix, os demandantes abrem um chamado para realizar a implementação. Então, uma nova Branch intitulada Hotfix-NumeroChamado é criada. Ao final, é realizado o merge com a Branch Main.
- Em caso de remediação de segurança, os demandantes abrem um chamado para realizar a implementação. Então, uma nova Branch intitulada Security-NumeroChamado é criada. Ao final, é realizado o merge com a Branch Main.
- Uma solução de análise estática de código é adotada nesta estratégia para cada cenário.

DESAFIO

Quais as possíveis implicações no contexto da análise estática de código, em termos de gerenciamento das vulnerabilidades encontradas, para este cenário, considerando um alto número de requisições pelos demandantes e **quais as suas recomendações?**



PUC Minas



PUC Minas

ANÁLISE ESTÁTICA NA IDE

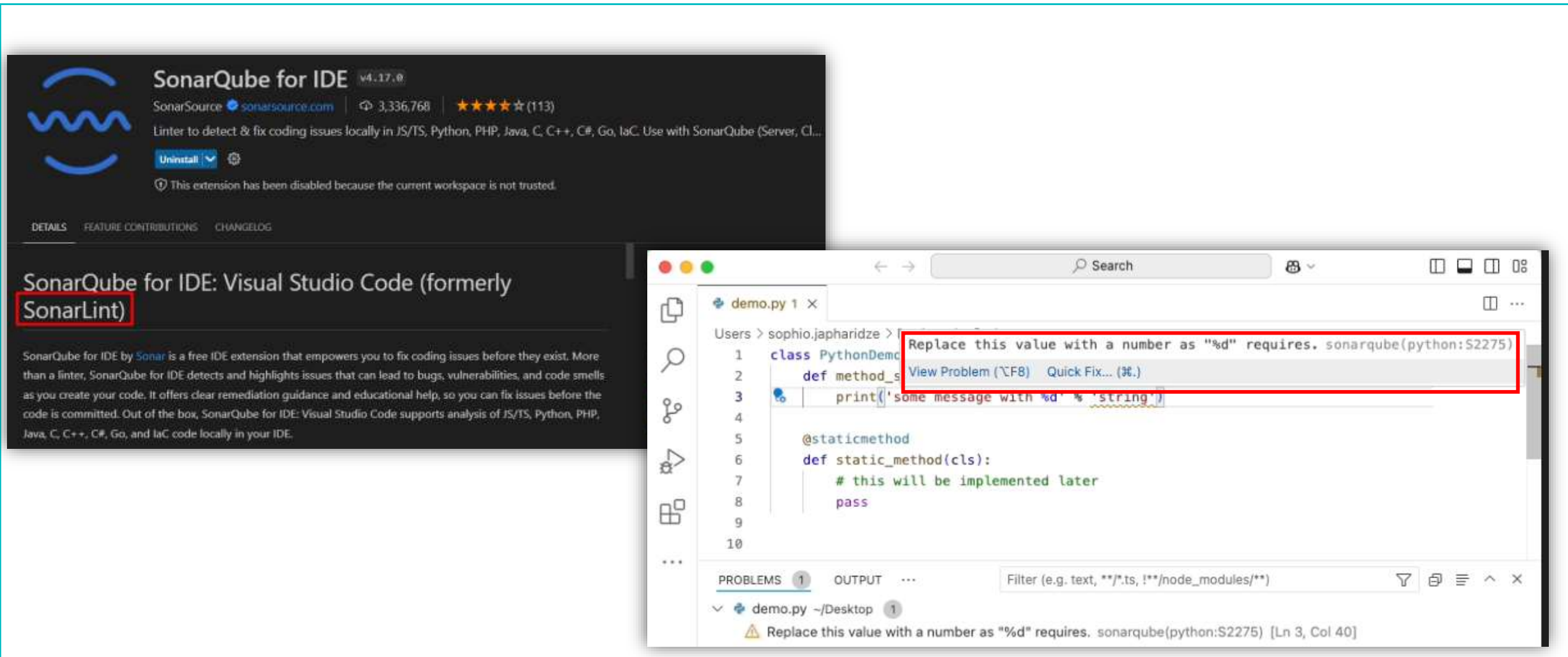
ANÁLISE ESTÁTICA DE CÓDIGO NA IDE

- Requer-se avaliar a compatibilidade com a IDE
- Requer-se avaliar **aspectos de licença e comerciais**
- Requer-se avaliar a necessidade de um produto integrado requerido
- Requer-se avaliar a **compatibilidade com a linguagem de programação** adotada
- **Automatizar a implantação do plugin de IDE** nos ativos dos colaboradores
- **Objetivo:** Antecipar correções de segurança durante a escrita do código

MOTIVAÇÕES

- Detectar **bugs** e **falhas de segurança** durante a escrita do código-fonte
- Feedback rápido
- Recomendações baseadas em melhores práticas
- Fornecimento de documentação apropriada para compreensão do problema e como resolver
- Permite encontrar problemas antigos no software que ainda não foram remediados

ANÁLISE ESTÁTICA DE CÓDIGO NA IDE



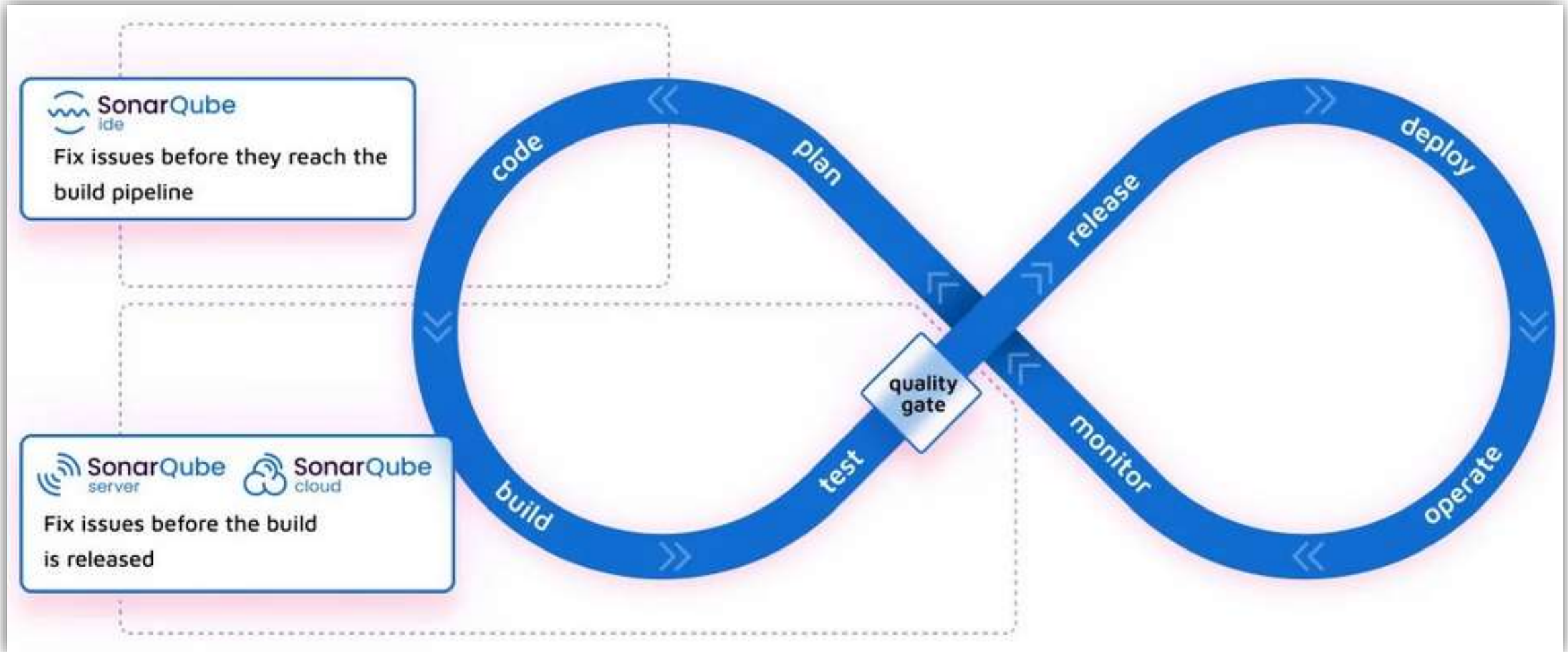
The image displays two screenshots related to static code analysis in an IDE.

The left screenshot shows the **SonarQube for IDE** extension page in Visual Studio Code. The extension is version 4.17.0, published by SonarSource, and has 3,336,768 downloads and 113 star ratings. It is described as a "Linter to detect & fix coding issues locally in JS/TS, Python, PHP, Java, C, C++, C#, Go, IaC. Use with SonarQube (Server, Cl...)". The extension is currently disabled with the message: "This extension has been disabled because the current workspace is not trusted." Below this, the text "SonarQube for IDE: Visual Studio Code (formerly SonarLint)" is highlighted with a red box.

The right screenshot shows a code editor window with a Python file named `demo.py`. The code defines a `PythonDemo` class with a `method_s` function and a static method `static_method`. A static analysis error is highlighted with a red box, indicating a format string issue: "Replace this value with a number as \"%d\" requires. sonarqube(python:S2275)". The error message includes a "View Problem" link and a "Quick Fix" button. The `PROBLEMS` panel at the bottom shows the error details: "demo.py ~/Desktop 1" and "Replace this value with a number as \"%d\" requires. sonarqube(python:S2275) [Ln 3, Col 40]".

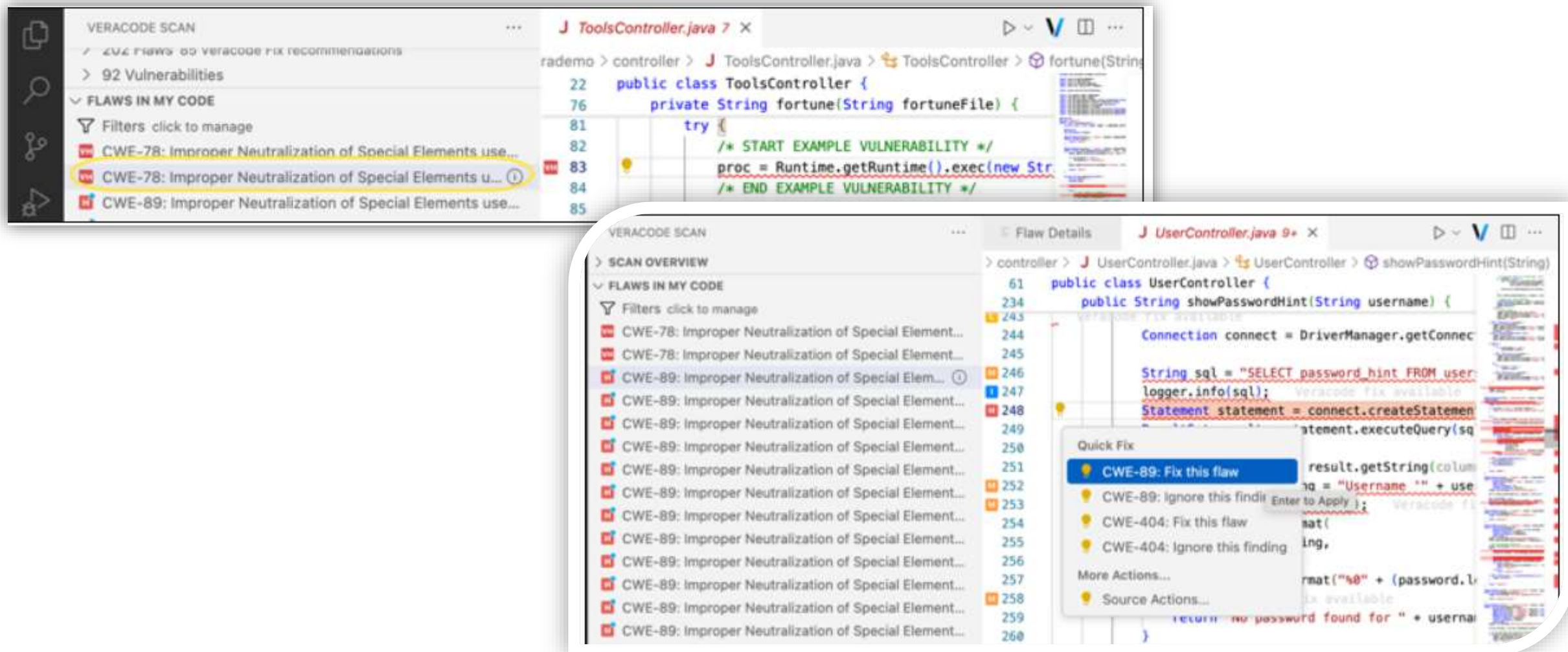
Fonte: <https://www.sonarsource.com/products/sonarlint/>

ANÁLISE ESTÁTICA DE CÓDIGO NA IDE



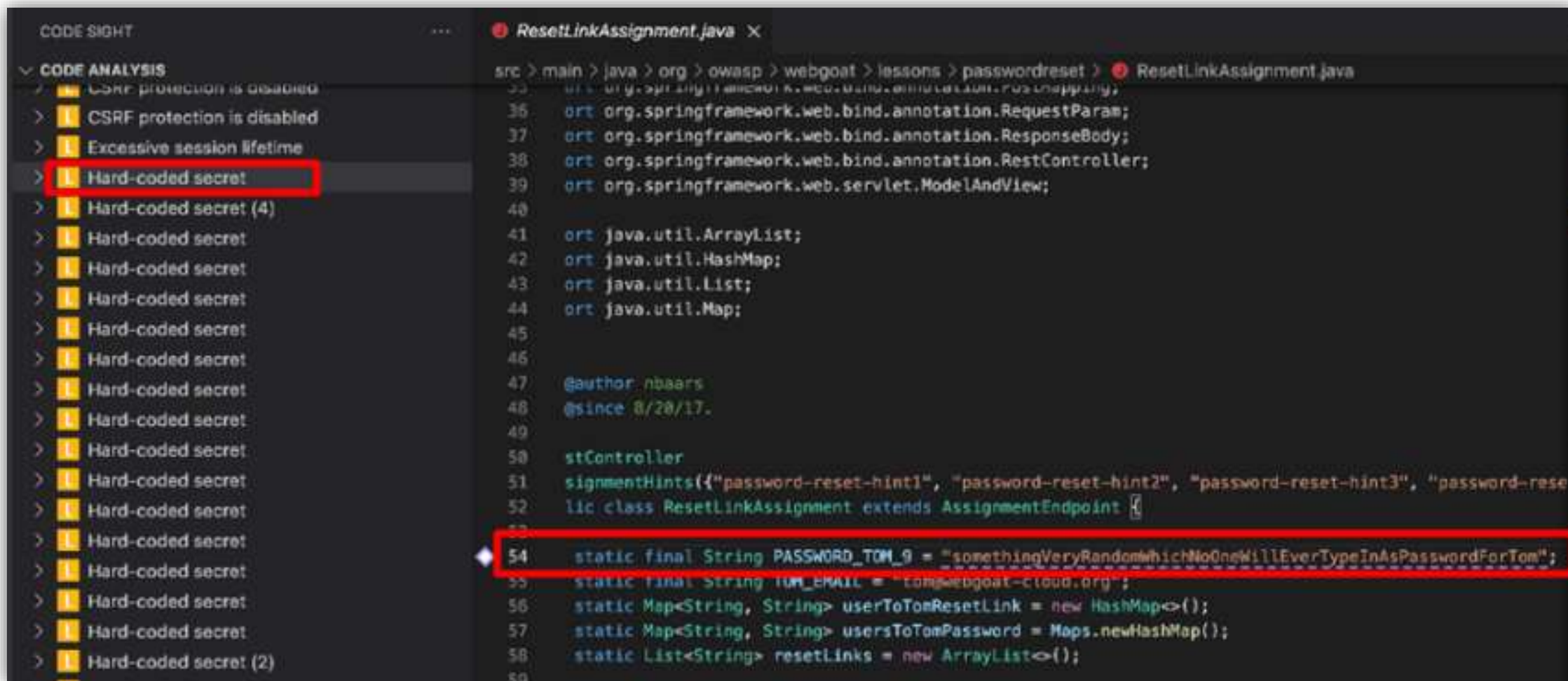
Fonte: <https://www.sonarsource.com/products/sonarlint/>

ANÁLISE ESTÁTICA DE CÓDIGO NA IDE



Fonte: https://docs.veracode.com/r/Veracode_Scan_for_VS_Code. Plugin para IDE Veracode Scan da Veracode.

ANÁLISE ESTÁTICA DE CÓDIGO NA IDE



Fonte: <https://www.blackduck.com/code-sight.html>. Plugin para IDE Code Sight, da Synopsys.

FERRAMENTAS PARA IDE

SYNOPSYS®

Checkmarx

sonarqube

VERACODE



snyk





PUC Minas

DESAFIO

DESAFIO

Uma empresa que iniciou sua cultura DevSecOps estava pretendendo adotar a verificação de análise estática de código iniciando o mais próximo dos desenvolvedores. A empresa contratou uma consultoria que conduziu este processo. A empresa possui aproximadamente 400 desenvolvedores. Houve uma comunicação interna seguida por uma implementação massiva nos computadores dos desenvolvedores. Inicialmente, a empresa obteve sucesso na implantação dos plugins de Linter na IDE, contudo, em estágios posteriores, as mesmas vulnerabilidades que foram identificadas na IDE de desenvolvimento estavam ocorrendo também. O sintoma era que os desenvolvedores não estavam corrigindo a vulnerabilidade durante a codificação.

Quais as possíveis causas e quais estratégias podem ajudar a resolver este problema?



PUC Minas



PUC Minas

GERENCIAMENTO DE SEGREDOS

GERENCIAMENTO DE SEGREDOS

- Informações sigilosas incorporadas ao código-fonte (ou hard-coded secrets) são fontes comuns de violação de dados
- Informações como secrets, chaves de criptografia, tokens de API, Chaves de SSH, Senhas, tokens OAuth, entre outros, sejam devidamente protegidos e não façam parte do código-fonte

GERENCIAMENTO DE SEGREDOS

```
src > main > java > org > owasp >  
38  import java.util.List;  
39  
40  /**  
41   * Created by jason on  
42   */  
43  
44  @RestController  
45  @AssignmentHints({"les  
46  public class SampleAtt  
47  
48      String secretValue  
49  
50      String mySSHKey = "-----BEGIN EC PRIVATE KEY----- Main Event: A secret, such as a  
51      MHcCAQEEIJWbvSW7h50HPwG+bWR3DXgQ6Yh0xYbe0ifr1rRUvsUuoAoGCCqGSM49  
52      AwEHoUQDQgAE34yHdT/dZ7hVi1XVCZZQUjMUtNR56CXUCjn9Aa0JEYBmfXvFf0qU  
53      KutYhIiNJgDAJqMgQZI8RnA80wGgrxcPxA==  
54      -----END EC PRIVATE KEY-----";
```

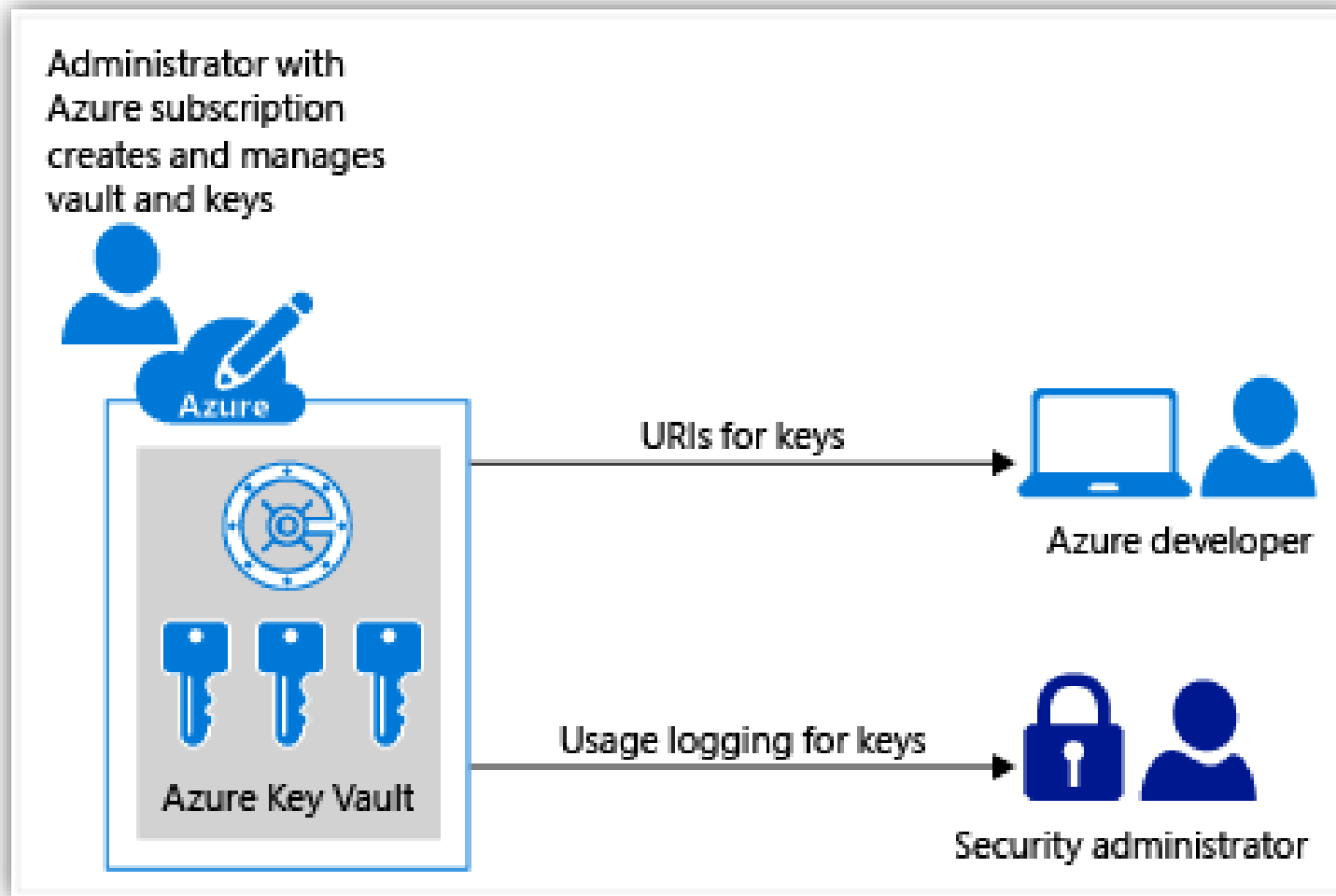
ISSUE: Hard-coded secret
Dismiss
Event 1 of 1
A secret, such as a password, cryptographic key, or token is stored in plaintext directly in the source code, in an application's properties, or configuration file. Users with access to the secret may then use the secret to access resources that they otherwise would not have access to. Secret type: SSH key.
Remediation:
Avoid setting sensitive configuration values as string literals. Instead, these values should be set using variables with the sensitive data loaded from an encrypted file or a secret store.
Checker: hardcoded_secret

Fonte: <https://www.blackduck.com/code-sight.html>. Plugin para IDE **Code Sight**, da Synopsys.

GERENCIAMENTO DE SEGREDOS

- Segredos não devem ser transmitidos de um estágio para outro no pipeline de CI/CD, onde não são necessários
- Apenas a pessoa ou máquina autorizada tenha acesso ao recurso de Vault
- Usar controle de acesso adequado e garantir que apenas o privilégio necessário seja atribuído a um usuário ou máquina
- Garantir mecanismos de backup da solução de Vault

GERENCIAMENTO DE SEGREDOS



Fonte: <https://learn.microsoft.com/pt-br/azure/key-vault/general/basic-concepts>

GERENCIAMENTO DE SEGREDOS

PAPÉIS E RESPONSABILIDADES

Desenvolvimento	Segurança	Operações
▪ Adotar bibliotecas para as linguagens de desenvolvimento para integração com o Vault ▪ Definir o controle de acesso ao Vault ▪ Reportar incidentes envolvendo hard-coded ▪ Identificar falsos-positivos (código de Teste)	▪ Desenvolver políticas para garantir que informações sigilosas não residem em código-fonte ▪ Propor melhores práticas do uso de Vault ▪ Adotar análise estática de código para auditar ocorrências de hard-coded e realizar análise de causa raiz ▪ Realizar a revisão de acesso ao Vault ▪ Realizar conscientização e treinamento	▪ Realizar a implantação de Vault ▪ Garantir que somente recursos de Cloud autorizados possuam acesso ao Vault ▪ Garantir que nenhum recurso de Cloud e Banco de Dados sejam acessados diretamente, somente com o token autorizado ▪ Provisionamentos automatizados ▪ Garantir meios de recuperação do Vault ▪ Reportar incidentes relacionados ao Vault

GERENCIAMENTO DE SEGREDOS NAS OPERAÇÕES

Operações

Acesso à Produção
Exclusivamente por
GMUD

Adotar Jump
Server

Adotar Cofre de
Senhas (sessão
monitorada e
gravada)

Modificações em
Produção
Exclusivamente
Automatizada

Infraestrutura
Imutável



PUC Minas

DESAFIO

DESAFIO

Uma empresa está trabalhando em um projeto envolvendo um forte foco na economia de API. As APIs irão adotar tokens JWT e, portanto, será necessário gerenciar diversos secrets, chaves de criptografia, entre outros. O projeto foi iniciado e durante uma análise estática de código foram identificados diversos problemas de hard-coded de informações sensíveis referentes ao projeto.

Qual a possível causa dos achados e quais as estratégias para resolver?



PUC Minas



PUC Minas

SOFTWARE COMPOSITION ANALYSIS

SCA

SOFTWARE COMPOSITION ANALYSIS

Perigo: componentes desatualizados em 90% do código

Da Redação

12/05/2020

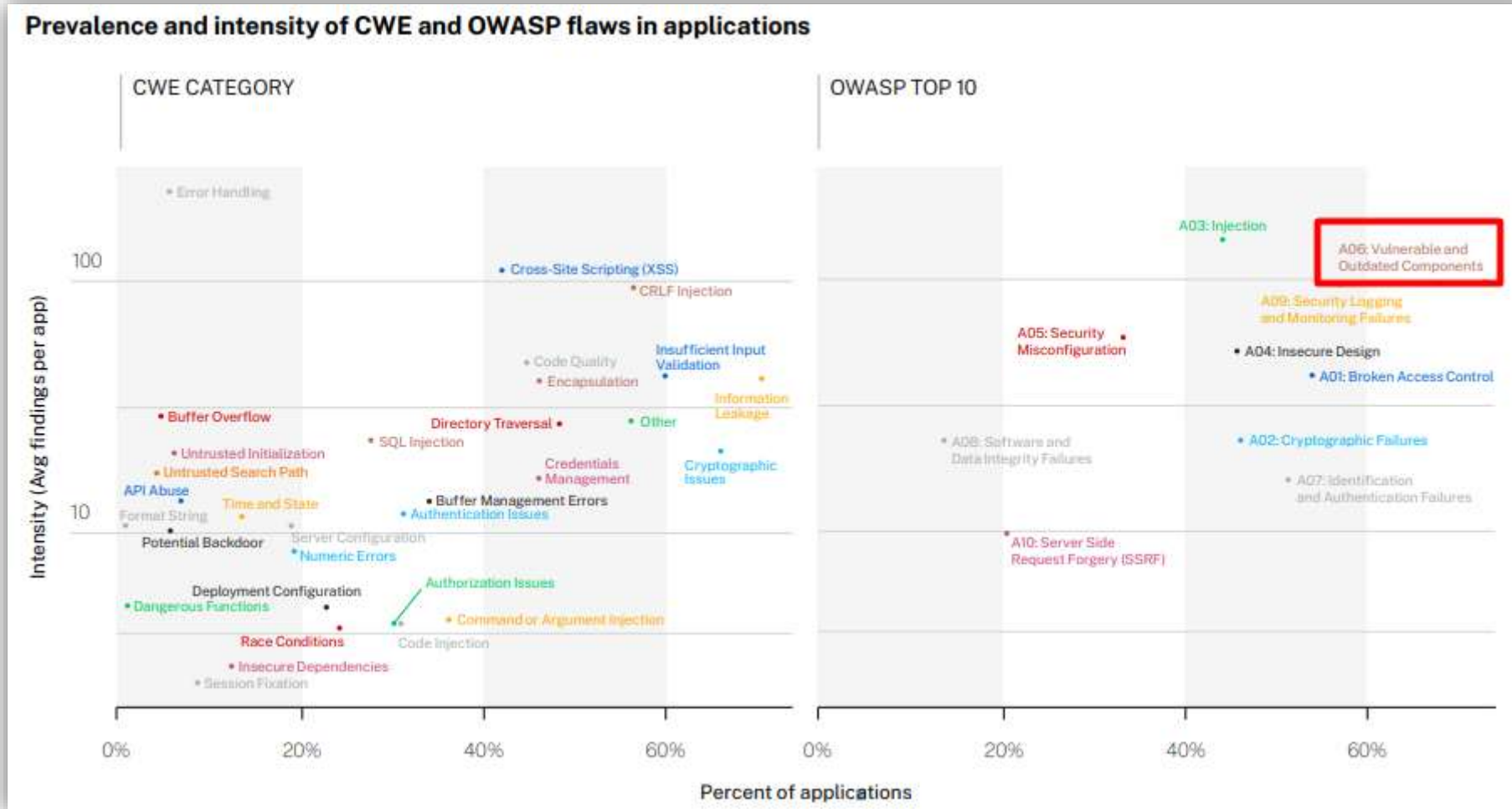
23:49

91% dos aplicativos usam pelo menos um componente open source sem atualização há quatro anos ou mais – ou que foi completamente abandonado e não tem mais desenvolvimento

O uso de componentes de software de código aberto continua a ser o padrão para o desenvolvimento, com uma média de sete em cada 10 linhas de código no aplicativo médio provenientes de um projeto de código aberto, mostra o estudo. Cada aplicação acaba utilizando, em média, 445 componentes de código aberto, um aumento de 49% em relação à pesquisa do ano passado. O setor de infraestrutura de Internet e software é quem mais usa as bases de código de código (83%), seguidas pelos fabricantes de dispositivos de Internet das Coisas (IoT). Enquanto isso, as telecomunicações e a indústria sem fio usam a menor quantidade de componentes de código aberto, com 46%, enquanto os setores de manufatura, sistemas de controle industrial e robótica utilizam em 50%.

Fonte: <https://www.cisoadvisor.com.br/perigo-componentes-desatualizados-em-90-dos-codigos>

SOFTWARE COMPOSITION ANALYSIS



Fonte: Veracode. State of Software Security 2025 Report.

SBOM

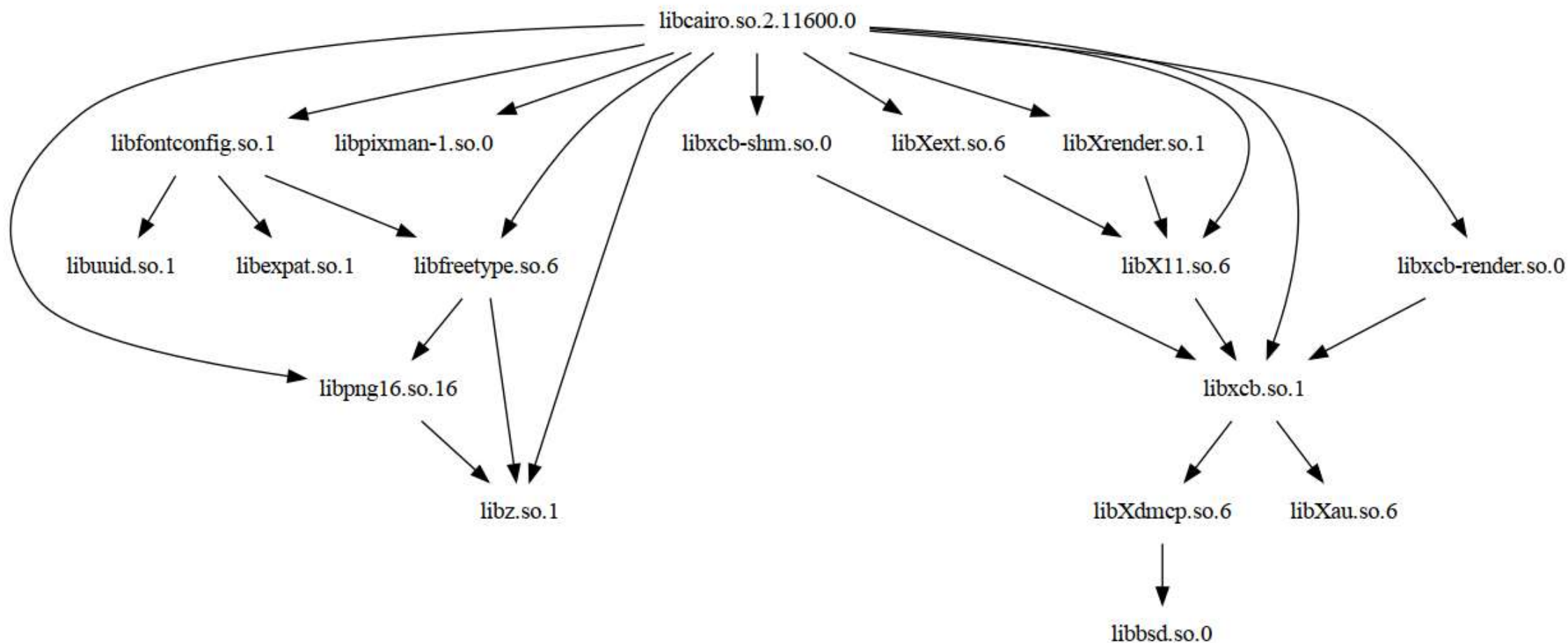


- Software Bill of Materials (SBOM) é um **inventário abrangente de todos os componentes de código-aberto** como parte dos artefatos de software, incluindo suas versões e dependências
- Soluções de **SCA** geram SBOM e administram automaticamente
- Suportam a detecção de vulnerabilidades, conformidade de licenças e auditorias de segurança

SOFTWARE COMPOSITION ANALYSIS

- Realiza um **inventário de todos os pacotes constituintes do software** por meio do SBOM
- SCA fornece uma análise profunda de pacotes de código aberto incorporados ao software
- Avalia vulnerabilidades e violações de conformidade quanto ao uso das licenças das dependências
- Geralmente usado com soluções de análise estática de código e **integrado ao Pipeline**

SOFTWARE COMPOSITION ANALYSIS



Fonte: <https://software-compliance.se/posts/2020-10-31-dependencies/>. **Bibliotecas transientes ou recursivas.**

SEMANTIC VERSION

1.2.3

Major Version

Break Change

Mudanças significativas, introdução de novas features, pode causar refatoração de código. Pode “quebrar” o software.

Minor Version

Non-Breaking

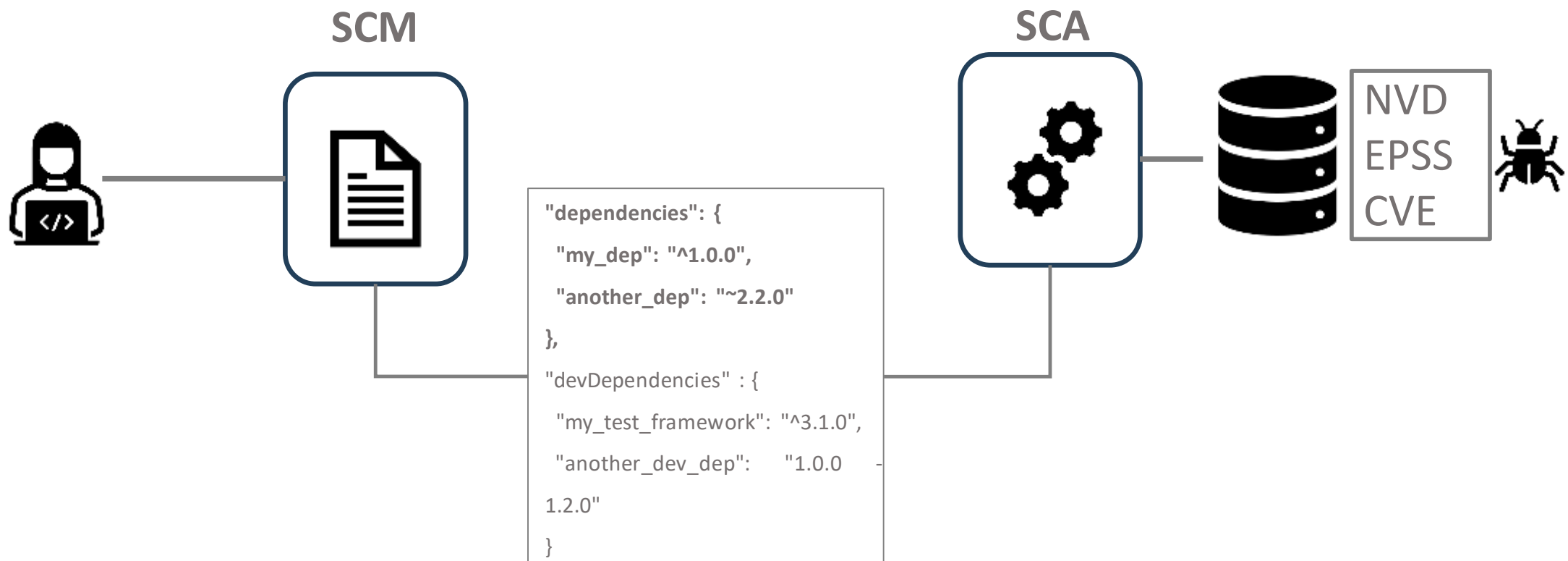
Adição de recursos ou funcionalidades, mantendo a compatibilidade. Geralmente não “quebra” o software.

Patch Version

Non-Breaking

Correções de bugs, patches ou pequenas atualizações sem introduzir recursos. Geralmente corrige vulnerabilidades.

SOFTWARE COMPOSITION ANALYSIS



FERRAMENTAS SCA

State	Exploitability ?	Risk Score	ID	Category	Identified in Package ?
✓	✖ Vulnerability	68 10 29 24 5			
To Verify	—	—	EPSS 93.2%	9.8 CVE-2022-29078	CWE-74 Improper Neutralization ... ejs @ 2.7.4 ^(T)
To Verify	—	—	—	9.8 Cx35ef42d7-054c	CWE-94 Improper Control of Gen... ejs @ 2.7.4 ^(T)
To Verify	—	—	EPSS <0.1%	9.8 CVE-2021-23436	CWE-1321 Improperly Controlled... immer @ 8.0.1 ^(T)
To Verify	—	—	EPSS 0.1%	9.8 CVE-2021-3757	CWE-1321 Improperly Controlled... immer @ 8.0.1 ^(T)
To Verify	—	—	EPSS 0.6%	9.8 CVE-2023-42282	CWE-918 Server-Side Request F... ip @ 1.1.9 ^(T)

Fonte: <https://checkmarx.com>

FERRAMENTAS SCA

Exploitable Path ?



No Exploitable Path Detected

Exploitable Path was enabled in this scan, but usage of functions that are in the exploitable path of the vulnerability were not detected in your project.



FERRAMENTAS SCA

SYNOPSYS[®]

VERACODE

Checkmarx



Dependabot



snyk

sonarqube



OWASP
Dependency-Check



PUC Minas



PUC Minas

DESAFIO

DESAFIO

Uma empresa preocupada com a adoção massiva de componentes de terceiros nos projetos de software, resolveu adotar uma ferramenta de SCA. Em um dos projetos críticos, foi observado duas vulnerabilidades críticas com CVSS 9.8 e amplamente sendo explorada. A empresa decide priorizar esta remediação, contudo há os seguintes:

- A versão vulnerável é a 2.4.6. A versão corrigida e recomendada é a 6.2.7.
- Uma biblioteca na versão 1.4.17 é considerada EOL/EOS.
- Não é possível saber se a aplicação inclui a porção de código vulnerável da biblioteca.

Quais os possíveis problemas a serem enfrentados e recomendações?



PUC Minas